

Pragmatic Angular: Building Dynamic Web Applications

Richard Krasso
First Edition

Table of Contents

<i>Chapter 1. Introduction to Angular</i>	3
Chapter Overview	3
Learning Objectives	3
File Management	3
Building a Base Layout in Angular	5
Building a Landing Page	10
Hands-On 1.1: Angular CLI	15
<i>Chapter 2. Routing</i>	16
Chapter Overview	16
Learning Objectives	16
Generating Components	16
Basic Routing	17
Routing in Action	33
Hands-On 2.1: Routing in Action	37
<i>Chapter 3. Error Handling and Unit Testing</i>	38
Chapter Overview	38
Learning Objectives	38
Error Handling	38
Unit Testing	41
Angular Template Syntax	46
Hands-On 3.1: Unit Testing	54
<i>Chapter 4. Angular Forms I</i>	55
Chapter Overview	56
Learning Objectives	56
Template-Driven Forms with TDD	56

Hands-On 4.1: TDD and Template-Driven Forms	66
Chapter 5. Angular Forms II	67
Chapter Overview	67
Learning Objectives	67
Reactive Forms with TDD	67
Hands-On 5.1: TDD and Reactive Forms	82
Chapter 6. Angular Forms III	84
Chapter Overview	84
Learning Objectives	85
Dynamic Forms and TDD	85
Hands-On 6.1: TDD and Complex Forms	94
Chapter 7. Input/Output Properties	95
Chapter Overview	96
Learning Objectives	96
Input/Ouptut Properties	96
Hands-On 7.1: TDD and Component Communications	107
Chapter 8. HTTP Client	108
Chapter Overview	108
Learning Objectives	108
HttpClient	109
Hands-On 8.1: TDD and HttpClient	113
Chapter 9. Modules and Deployment	115
Chapter Overview	115
Learning Objectives	115
GitHub Pages Deployment	115

Chapter 1. Introduction to Angular

Chapter Overview

In this chapter, we will take a look at the Angular ecosystem, a fast, opinionated, web framework used to build Single Page Applications (SPAs). We will begin our journey by exploring what Angular is, how to generate new components using Angular’s CLI tool, and how to build a landing page using the Angular framework.

All examples and programs in this book have been created using Angular 18 with Node.js 20. Angular rolls out a new stable version approximately every six months. Node.js, on the other hand, releases a new stable version approximately once a year. Angular is very strict on which versions of Node.js it supports. Each new release of Angular is considered “active” for 6 months and then it moves into Long-Term Support (LTS) for an additional 12 months. This means, that while a version may no longer be considered “active,” it is still supported for another 12 months. Always refer to Angular’s official documentation for [Angular versioning and releases](#). Despite my best efforts, to keep pace with the latest’s version of Angular, is not always achievable. Therefore, it is possible that by the time you read this, a newer version of Angular would have been released. Consequently, the only way to guarantee that the code examples and hands-on projects in this book work correctly, is to stick with the versions used in this book.

Learning Objectives

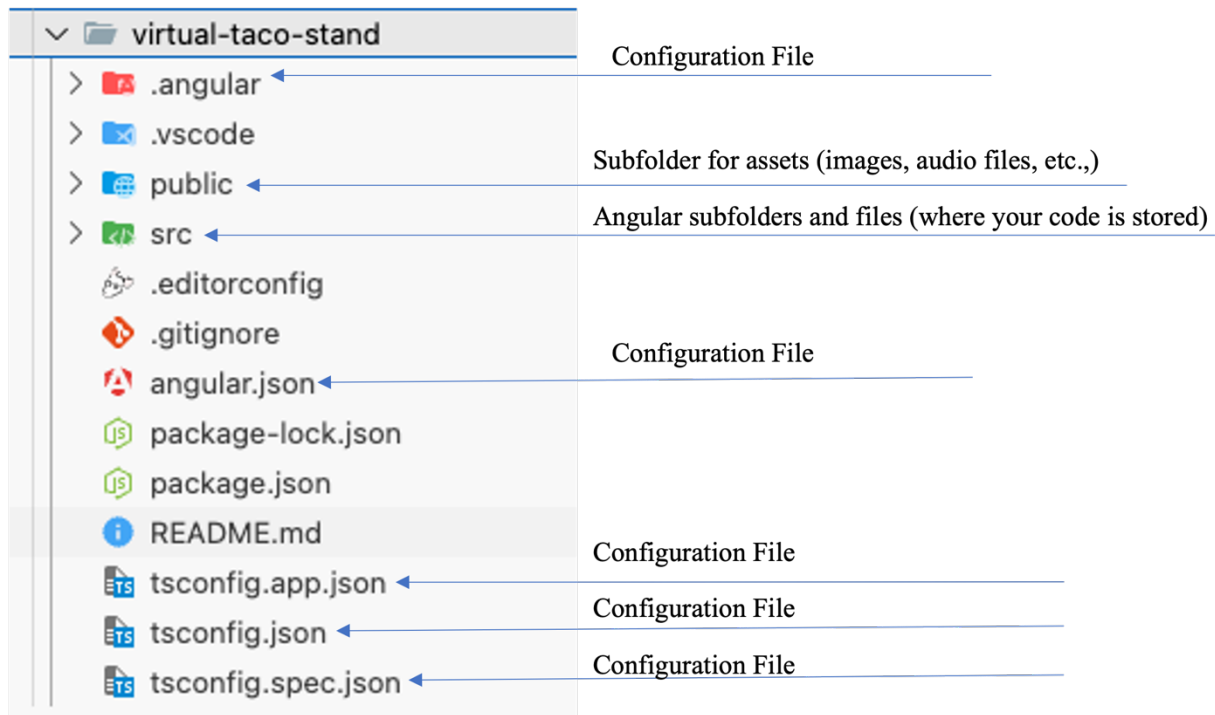
By the end of this chapter, you should be able to:

- Examine the Angular ecosystem and its versioning strategy.
- Analyze Angular project setup with Angular 18 and Node.js 20.
- Build a basic SPA layout using an Angular component.
- Generate a HomeComponent with inline styling and HTML using Angular’s CLI.

File Management

Angular projects use several types of files and folders to manage the organization of a project. At first glance, the sheer number of files and folders can be intimidating and overwhelming to new developers. However, most of the files are considered configuration files and are largely responsible for the managing of Angular’s vast ecosystem. It is important that before you begin working on an Angular project that you understand what each of these files and folders are responsible for. Otherwise, if you run into versioning or compatibility issues, you may not know how to resolve them. Just like in Node.js, Angular uses NPM to manage dependencies and a compiler named Ivy to manage the transpiling of TypeScript code into native HTML, CSS, and JavaScript. TypeScript is a superset of JavaScript, meaning, you write code in TypeScript and it is transpiled into native JavaScript. TypeScript is a very popular language and is gaining a lot of momentum in the JavaScript community. At its core, TypeScript is nothing more than a wrapper that sits on top of JavaScript enabling you to define “types,” which is more in-line with traditional object-

oriented programming languages like C++, C#, Java, and Python. The fact that it resembles many of the features seen in traditional object-oriented programming languages, makes it the preferred choice for object-oriented developers wanting to learn native JavaScript.



The **src** folder is a subfolder with more subfolders and files. Each of these files and subfolders are where new components, services, directives, etc., are placed once they are generated using Angular's CLI. All of the work you will complete to build a functional Angular project will be placed in this folder. The **public** folder is where you store website assets like images, audio files, data files, etc., If you want to display an image in your Angular project, this is where you will store it. The **package.json** file is where the required dependencies for the project are stored. And, the **angular.json** file is where the required configurations for your project are stored. Dependencies include, third-party JavaScript and CSS packages, compiler configurations, and build configurations and budgets, and test configurations.

By default, Angular's CLI tool, configures the following package.json scripts:

Key	Value	Comments
ng	ng	Enables running Angular CLI commands directly through npm.
start	ng serve	Executes the build process and deploys the

		project to a development web server (starts the development server).
build	ng build	Executes the build process. Compiles the Angular project into an output directory.
watch	ng build --watch --configuration development	Executes the build process and watches for file changes.
test	ng test	Executes the Karma and Jasmine unit tests.

Each of these commands are executed through the CLI in the directory where the project is saved. For example, if I wanted to start the virtual-taco-stand project, I would need to make sure I was in the virtual-taco-stand directory before running: **ng serve**. Optionally, I could right-click the solution in VS Code and select “Open Integrated Terminal,” which will open VS Code’s build-in terminal in the directory where it was selected.

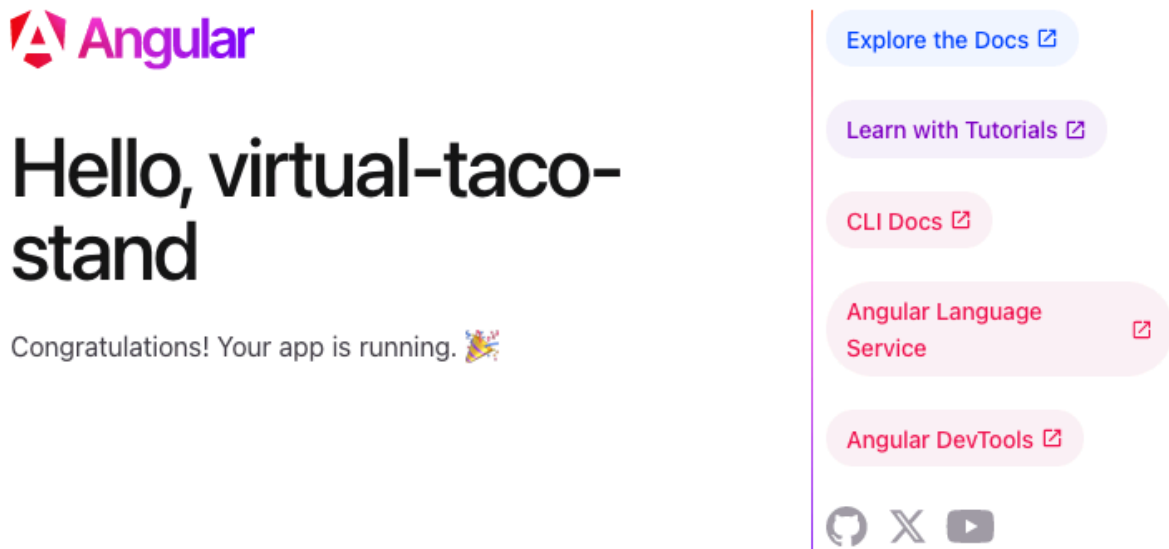
Building a Base Layout in Angular

In the upcoming chapters of this book, we will take you on a journey through the creation of the Virtual Taco Stand project – a web application built with Angular. This platform is designed for culinary enthusiasts eager to discover new taco menus, place orders, and check out the latest taco specials. Each chapter will introduce new features and functionalities, ranging from simple navigation to complex form management and user authentication, providing an in-depth exploration of the core features in the Angular ecosystem. The Virtual Taco Stand project is born out of a love for tacos and the ambition to craft a virtual environment where users can experience the charm of their preferred taco stands. By the end of this book, you’ll have crafted a dynamic and user-friendly Angular web application.

In this section, we will explore how to build a base layout for the Virtual Taco Stand web application. The starter project for the Virtual Taco Stand web application is located in the data files folder.

1. Open the virtual-taco-stand project in your code editor.
2. Open a new terminal window (right-click the solution and choose Open Integrated Terminal).

3. Run the npm install command (npm install).
4. Start the application (ng serve).
5. Open a new browser window and navigate to: http://localhost:4200.
6. You should see Angular's default landing/welcome page.



7. Stop the server by going back to the open terminal and entering: Ctrl + C.
8. Open the app folder (located under the src folder) and delete the following files:
 - a. app.component.html
 - b. app.component.css
9. Open the app.component.ts file and update it to the following:

```
import { Component } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="wrapper">
      <header class="banner">
```

```

    
  </header>

```

```

<main class="main-content">

```

```

  <nav class="navbar">
    <ul>
      <li><a routerLink="/">Home</a></li>
      <li><a href="#">Menu</a></li>
      <li><a href="#">Order</a></li>
      <li><a href="#">Daily Specials</a></li>
      <li><a href="#">Feedback</a></li>
    </ul>
  </nav>

```

```

  <section class="content">
    <router-outlet />
  </section>
</main>

```

```

<footer class="footer">
  <nav class="footer-nav">
    <a routerLink="/">Home</a> |
    <a href="#">Menu</a> |
    <a href="#">Order</a> |
    <a href="#">Daily Specials</a> |
    <a href="#">Feedback</a>
  </nav>
  <p>&copy; 2024 Virtual Taco Stand</p>
</footer>

```

```

</div>

```

```

`
,

```

```

styles: [
`
,

```

```

`

```

```

]
})
export class AppComponent {
}

```

The images for this component are located in the data-files folder, in a subfolder named assets. These files will need to be copied over to your projects assets folder,

which is located under the public folder. In the above code, we are using inline HTML templates and CSS to build a base layout for the Virtual Taco Stand project. The HTML element `<router-outlet />` is used as a placeholder for the content to display from other components in the application. Effectively, we are saying, use this section to display the HTML content from other components in our application. This is the basis of how we create layouts in an Angular application. We place the `<router-outlet />` element in the section where we want other components to be displayed.

10. Open the styles.css file (located under the src folder) and add the following CSS styles:

```
/* Import Google Fonts */  
@import  
url('https://fonts.googleapis.com/css2?family=Lato:wght@400;700&family=Merriweather:wght@400;700&family=Montserrat:wght@700&display=swap');  
  
body, header, nav, main, section, footer, img, figure, figcaption, div, ul {  
  margin: 0;  
  padding: 0;  
  border: 0;  
}  
  
body {  
  height: 100%;  
}  
  
.wrapper {  
  display: flex;  
  flex-direction: column;  
  min-height: 100vh;  
  margin: 0 auto;  
  width: 75%;  
  background-color: #FFFFFF;  
  color: #000000;  
}  
  
.banner {  
  background-color: #000000;  
  color: #FFFFFF;  
  text-align: center;  
}  
  
.banner-img {
```

```
max-width: 100%;  
height: auto;  
}
```

```
.main-content {  
  display: flex;  
  flex: 1;  
}
```

```
.navbar {  
  flex: 0 0 200px;  
  padding: 20px;  
}
```

```
h1, h2, h3, h4, h5, h6 {  
  font-family: 'Montserrat', sans-serif;  
}
```

```
.navbar ul {  
  list-style-type: none;  
  padding: 0;  
  margin: 0;  
}
```

```
.navbar li {  
  padding: 8px;  
  transition: background-color 0.3s ease, color 0.3s ease;  
  font-family: 'Lato', sans-serif;  
}
```

```
.navbar li:hover {  
  background-color: #F2F2F2;  
  color: #0056B3;  
  cursor: pointer;  
}
```

```
.navbar a {  
  text-decoration: none;  
  color: inherit;  
}
```

```
.content {  
  flex: 1;  
  padding: 20px;
```

```

    font-family: 'Merriweather', serif;
}

.footer {
    background-color: #000000;
    color: #FFFFFF;
    text-align: center;
    padding: 20px;
    font-family: 'Lato', sans-serif;
}

.footer a {
    color: #FFFFFF;
    text-decoration: none;
}

.footer a:hover {
    text-decoration: underline;
}

```

These styles will form the foundation for our application's layout.

11. Open a new terminal window and start the project to test the changes (ng serve). You should see a navigation bar and footer. If the terminal window shows errors, it is likely you have a syntax error somewhere in your code. Go back through the steps again to make sure there are no errors in your HTML or CSS files.

Building a Landing Page

Generally speaking, there are several ways to generate a component in Angular. One approach is to generate a new component with four files, which includes `.component.ts`, `.component.html`, `.component.css`, and `.component.spec.ts`. Using this approach, all of your HTML code is placed in the file with a `.html` extension. Conversely, all of your CSS code is placed in the file with an `.css` extension and all of your TypeScript code is placed in the file with a `.ts` extension. Another approach is to generate a new component with inline HTML templates and CSS styling. The merits of each approach are debatable and outside the scope of this book; however, for all of the remaining examples in this book we will be using inline HTML templates and CSS styling (a single file) for our components.

Continuing with where we left off in the previous section of the virtual-taco-stand project:

1. Open the project in your editor.
2. Open a new terminal window and generate a new component named "home" using the following command:

ng generate component home -s -t

The flags -s and -t tell the Angular CLI to generate a component named “home” with inline templates (HTML) and inline styles (CSS). If you omit the flags, it will generate a component with four files: home.component.ts, home.component.html, home.component.css, home.component.spec.ts. If this happens, simply delete the files and rerun the generate command.

3. Open the home.component.ts file and add the following code:

```
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-home',  
  standalone: true,  
  imports: [],  
  template: `  
    <div>  
      <h1>Virtual Taco Stand</h1>  
      <h2>The Virtual Taco Stand is located in the heart of downtown!</h2>  
      <p>  
        We specialize in good-ole home cooking. Our menu ranges from Tacos Al  
Pastor  
        to our famous Birra Tacos. We also have a variety of daily specials that are  
        sure to please your taste buds. We are open 24 hours a day, 7 days a week.  
        Come on down and see us!  
      </p>  
      <p>  
        Having a party? We cater! Give us a call and we will be happy to help you  
        with your catering needs.  
      </p>  
  
      <div class="highlights-container">  
        <div class="highlight">  
            
          <p>  
            Discover the heart of downtown flavor at our taco stand! Fresh, vibrant,  
and bursting with authentic Mexican cuisine, our tacos are a downtown must-  
try.  
            Where every bite tells a story of tradition and taste.  
          </p>  
        </div>  
      </div>  
    </div>  
  `
```

```

</div>

<div class="highlight">
  
  <!-- give me an item description for a guy standing at the standing selling
tacos to customers-->
  <p>
    Welcome to our vibrant taco stand, where every bite is a fiesta!
    Join us for an unforgettable culinary journey, crafted by our passionate
taco artisans.
    Dive into the freshest flavors in town, where each taco promises a blend
of tradition and innovation.
    Your taste adventure starts here!
  </p>
</div>

<div class="highlight">
  
  <p>Feast your eyes on our trio of tacos, a vibrant homage to authentic
Mexican cuisine.
    Each taco is a masterpiece, carefully crafted with the freshest ingredients
and bursting with flavors that transport you straight to the heart of Mexico.
    Perfect for sharing (or indulging solo), these tacos are a celebration of
tradition, taste, and the art of good eating.
    Join us and savor the true essence of Mexican culinary excellence, one
taco at a time.</p>
</div>
</div>
</div>
\,
styles: [
\,
  .highlights-container {
    display: flex;
    justify-content: space-between;
    align-items: flex-start;
    flex-wrap: wrap;
    gap: 20px;
  }

```

```

    .highlight {
      text-align: center;
      flex: 0 1 calc(33.333% - 20px);
      box-sizing: border-box;
    }

    .highlight img {
      max-width: 100%;
      height: auto;
      object-fit: cover;
    }

    .highlight p {
      margin-top: 10px;
    }
  ],
})
export class HomeComponent {

}

```

In this code example, there are sections for “styles” and “template.” All of the HTML code you write is placed in the template section, wrapped with back ticks. And, all of the CSS code you write is placed under the styles section, wrapped in back ticks.

4. Open the app.routes.ts file and add a route for the HomeComponent.

```

import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';

export const routes: Routes = [
  {
    path: '',
    component: HomeComponent
  },
  {
    path: 'home',
    component: HomeComponent
  }
];

```

In this code example, a Routes array is defined, containing two objects. Both objects have a property for “path” and “component.” The “path” property specifies

the URL pattern triggers the rendering of the components HTML code. For example, if a user were to enter `http://localhost:4200` or `http://localhost:4200/home`, Angular knows to trigger the HomeComponent's HTML code. To understand how the "path" property works, consider the following: imagine you are in a large garden with many different sections, like a rose section, a vegetable section, and a pond area. Each section has its own path leading to it from the gardens entrance. When you enter the garden, you decide where you want to go by choosing one of these paths. In this scenario, the garden is like a website, and each section of the garden is like a different page on the website. The paths in the garden are similar to the "path" property in the routes of a website.

- a. The first path in the `app.routes.ts` file is an empty string. This is like the main entrance to the garden. When you enter, if you do not choose any specific path and just stay at the entrance, you are in the default section, which in the virtual-taco-stand application is represented by the HomeComponent or `http://localhost:4200`.
 - b. The second path is "home". This is like a sign in the garden pointing to the "Home" section. If you follow this path from the entrance, it takes you to the same welcome area as before. It's just another way to reach the same place, the HomeComponent.
5. To ensure the landing page (HomeComponent) is working correctly, run the serve command (`ng serve`) from the projects root folder.

VIRTUAL TACO STAND

Home

Menu

Order

Daily Specials

Feedback

Virtual Taco Stand

The Virtual Taco Stand is located in the heart of downtown!

We specialize in good-ole home cooking. Our menu ranges from Tacos Al Pastor to our famous Birra Tacos. We also have a variety of daily specials that are sure to please your taste buds. We are open 24 hours a day, 7 days a week. Come on down and see us!

Having a party? We cater! Give us a call and we will be happy to help you with your catering needs.



Discover the heart of downtown flavor at our taco stand! Fresh, vibrant, and bursting with authentic Mexican cuisine, our tacos are a downtown must-try. Where every bite tells a story of tradition and taste.



Welcome to our vibrant taco stand, where every bite is a fiesta! Join us for an unforgettable culinary journey, crafted by our passionate taco artisans. Dive into the freshest flavors in town, where each taco promises a blend of tradition and innovation. Your taste adventure starts here!



Feast your eyes on our trio of tacos, a vibrant homage to authentic Mexican cuisine. Each taco is a masterpiece, carefully crafted with the freshest ingredients and bursting with flavors that transport you straight to the heart of Mexico. Perfect for sharing (or indulging solo), these tacos are a celebration of tradition, taste, and the art of good eating. Join us and savor the true essence of Mexican cuisine.

You should see the websites landing page (HomeComponent), with three images (located in the data-files/assets folder), a navigation menu, a header, and a footer. If you do not see what's depicted in the image above or if there are errors in the terminal window, back trace your steps to ensure you do not have a typo somewhere in your code or a syntax error.

Hands-On 1.1: Angular CLI

In this hands-on project, you will be embarking on an exciting journey of building your own Angular application called “rpg-character-builder.” This application is designed to challenge your Angular skills by creating an immersive web application that caters to gamers and RPG enthusiasts. The focus here is on engagement, character customization, and interactive storytelling.

The idea behind the rpg-character-builder application was inspired by the love of video games and a desire to create a platform where users can manage their own collection of characters. Whether you are an avid gamer, RPG enthusiast, or simply someone looking to practice your Angular skills, rpg-character-builder is designed to cater to your needs.

For the first hands-on assignment, you will be setting up the rpg-character-builder project and creating the applications landing page (HomeComponent).

Instructions:

1. Use your code editor to go to the rpg-character-builder project (located in the data-files folder).
2. Design and build the applications base layout in the app.component.ts file. Include styling to bring the application to life. At minimum, your base layout should include a navigation menu (use the hashtag symbol ‘#’ as a placeholder for the hrefs), a main content area (place the router outlet here), a footer with another navigation menu (mirror your other navigation menu), and styling with at least 3 different font styles.
3. Generate a new component named “home.” Design and build the applications landing page (HomeComponent). Include styling, images, and varied fonts to bring the application to life. The goal here is to build a functional landing page, fully customized with styling, images, and textual content. Be creative, detailed, and build something that you would be proud to show off to a colleague or family member. At minimum, there should be at least 250 words of textual content (no Lorem Ipsum) and at least 3 images.
4. Add two new routes in app.routes.ts file for the HomeComponent. The first route should have a path of an empty string and the second route should have a path of “home.”

5. Run and test the application to ensure the landing page displays the appropriate content and there are no errors in the terminal window.

Grading:

You will earn 20 points for each of the following:

- Application executes without errors (no errors are present in the terminal window).
- Correct implementation of the application's base layout: header, navigation menu, main content, and footer.
- Correct implementation of the landing page (HomeComponent), with a minimum of three images and 250 words of textual content (no Lorem Ipsum).

This assignment is worth a total of 60 points. If two tasks are completed correctly, you will earn 40 points.

Hints:

- For setting up the and testing the application, remember to run: `ng serve`.
- To view the application in a browser, use the URL `http://localhost:4200`.
- To generate a new component use: **`ng generate component home -t -s`**.

Chapter 2. Routing

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Analyze the setup and configuration of basic routing in an Angular application.
- Examine the process of generating components with inline HTML and CSS for routing.
- Evaluate the implementation of navigation links in the `app.routes.ts` file.
- Assess the user experience improvements achieved through Angular routing.

Generating Components

In this section, we will take a look at how to generate components using Angular's CLI tool. All of the components generated in this section will use inline styling and HTML templates. That is, we will not be using a separate HTML and CSS file for the page's layout and styling.

Starting with where we left off from the virtual-taco-stand project in Chapter 2:

1. Open the project in your code editor.

2. Generate the following components: daily-specials, feedback, menu, order, and signin.
 - a. `ng generate component daily-specials -t -s`
 - b. `ng generate component feedback -t -s`
 - c. `ng generate component menu -t -s`
 - d. `ng generate component order -t -s`
 - e. `ng generate component signin -t -s`

The commands in step two will generate five new components with inline templates (-t) and inline styling (-s). As we work through the chapters in this book, we will add content and functionality to each of these components. For now, our goal is to simply generate the components using Angular's CLI.

In the next section we will take a look at how to navigate between components in an Angular project.

Basic Routing

In this section, we will explore the basic features and functionality of the Angular routing system. Routing is the backbone of how users navigate between web pages on your website. In Angular, routing is achieved by defining routes in the `app.routes.ts` file. Once a route is defined and registered, Angular will know where to look for resources when a user selects a link or enters a URL on your website. Think of routes as highways and intersections on a map. When you want to travel to a different location, you typically use a map for directions. For example, let's say we want to travel from Omaha, Nebraska to Chicago, Illinois. Using a map, we know taking I-80 East will take us to Chicago. In Angular, the same principles apply. A route tells Angular where a resource is located in your project. For example, let's say I had a "Contact Us" link on my website. If I wanted to display the "Contact Us" page, I would need to define a route in the `app.routes.ts` file, which provides Angular with instructions on how to associate the `ContactComponent` with the "Contact Us" link. Effectively, it provides a "mapping" or "bridge" between the link and resource in your Angular project.

Components are a term used in Angular to provide a structure for organizing your application into manageable parts. Imagine you are creating a garden. In this garden, you want different sections: one for flowers, one for vegetables, and another for fruits. Each of these sections can be thought of as a "component" in Angular. Just as each section in your proposed garden focuses on a different type of plant, each Angular component focuses on

different feature or section in your application. In gardening, before you plant anything, you need a plan or blueprint for the layout and organization of each section in your garden. In Angular, the blueprint (see our gardening example) is the component class. This class defines properties (like the types of flowers or vegetables) and methods (like watering or fertilizing the plants). Components use templates to define how the component will look (styling) and how data is displayed (HTML). Here is an example of a tree structure that represents our gardening example:

GardenApp (Main Application)

```
|
|-- FlowerSectionComponent (Manages the flower section)
|   |-- RoseComponent (Displays roses)
|   |-- TulipComponent (Displays tulips)
|
|-- VegetableSectionComponent (Manages the vegetable section)
|   |-- CarrotComponent (Displays carrots)
|   |-- LettuceComponent (Displays lettuce)
|
|-- FruitSectionComponent (Manages the fruit section)
|   |-- AppleComponent (Displays apples)
|   |-- OrangeComponent (Displays oranges)
```

In this example, we have our main application (GardenApp) and components for the flower, vegetable, and fruit sections. Next, each individual section has child components that represent the types of plants in the parent sections. While this is a relatively simple example, it should illustrate how components are used in Angular to define the structure of your application. Now, let's compare this structure to the structure of a traditional static HTML website:

GardenWebsite (Entire Website)

```
|
|-- index.html (Main page that might include links to other sections)
|
|-- flowers.html (Page for flower section)
|   |-- Roses and tulips are directly included in the HTML content
|
|-- vegetables.html (Page for vegetable section)
|   |-- Carrots and lettuce are directly included in the HTML content
|
|-- fruits.html (Page for fruit section)
|   |-- Apples and oranges are directly included in the HTML content
```

In this example, our static HTML and CSS website has an index.html file for the main page and separate pages for each section of the garden. Child flower types are included in the

separate section pages. The distinction between these two strategies is one takes a modularity approach to structuring the application (like `RoseComponent` or `CarrotComponent`). The benefit of this approach is sections can be developed, tested, and maintained independently. Whereas, in our `GardenWebsite` example, content is directly embedded within each page, making testing, maintenance, and reuse more challenging. The next significant advantage is in Angular, components are loaded dynamically, which means they are rendered without reloading the entire page. In our `GardenWebsite` example, there are no built-in mechanisms for updating content dynamically; it requires an entirely new page to be selected and loaded. The final significant advantage is data binding. Angular allows you to define variables, methods, and coding expressions that can be accessed directly in the view (HTML). Changes in the data are automatically reflected in the view and vice versa. In our `GardenWebsite` example, displaying dynamic data and interacting with the view (HTML) requires complex JavaScript coding.

Let's take a look at an example project that demonstrates some of the basic functionality of Angular routing. The solution for this project is located in the `data-files` folder.

1. Generate a new Angular project or use the starter project from the `data-files` folder.
 - a. New project command: **`ng new basic-routing`**
2. Select CSS for the styling engine and "n" for "server-side rendering."
3. Delete the `.html` and `.css` files for the app component (`app.component.css` and `app.component.html`).
4. Open the `app.component.ts` file and add the following code:

```
import { Component } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';
```

```
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="container">
      <header>
        <h1>{{ title }}</h1>
      </header>

      <nav>
        <ul>
          <li>
            <a routerLink="/contact">Contact Us</a>
```

```

    </li>
    <li>
      <a routerLink="/about">About Us</a>
    </li>
    <li>
      <a routerLink="/services">Our Services</a>
    </li>
  </ul>
</nav>

<main>
  <router-outlet />
</main>

<footer>
  &copy; {{ year }} Basic Routing Example <br />
  Professor Krasso
</footer>
</div>

```

```

`,
styles: `
.container {
  display: flex;
  flex-direction: column;
  min-height: 100vh;
  width: 75%;
  margin: 0 auto;
}

header {
  background-color: black;
  color: white;
  padding: 10px;
  text-align: center;
}

main {
  flex: 1;
}

nav {
  background-color: #f8f8f8;
  padding: 10px;

```

```

    text-align: center;
}

ul {
  display: block;
  list-style: none;
  padding: 0;
  margin: 0;
}

li {
  display: inline;
  margin-right: 10px;
}

footer {
  margin-top: 20px;
  height: 50px;
  padding: 10px;
  background-color: black;
  color: white;
  text-align: center;
}
,
})
export class AppComponent {
  // variables to be used in the template
  // scope is limited to the class AppComponent
  title: string;
  year: number;

  constructor() {
    this.title = 'Basic Routing Example'; // assign the title property a value of
    'Basic Routing Example'
    this.year = new Date().getFullYear(); // assign the current year to the year
    property using the Date object
  }
}

```

In this code example, we are using the app.component.ts file as the ‘Base Layout’ for our website. There are a few things going on here.

- a. The <h1> element in the HTML template is using “interpolation,” which is a template syntax that Angular uses to bind data from the component class to

the HTML code. The 'title' variable is declared in the "AppComponent" class as a class variable and it's assigned a value in the body of the classes' constructor.

- b. Another variable is declared in the "AppComponent" class and assigned a value from the call to `new Date().getFullYear()`. This is a built-in JavaScript function that returns the current year as a numerical value, from JavaScript's date object.
 - c. It is important to understand that Angular uses "interpolation" as a mechanism that allows us to declare, assign, and work with variables and objects in our HTML template. This power and flexibility allow us to create dynamic, fully functional web applications. Most of the standard functionality found in the JavaScript programming language is accessible through interpolation. This includes: variable assignment, conditions, iterators, etc.,
 - d. `routerLink` is an Angular built-in directive that enables navigation (routing) between resources in your Angular application. The syntax is `routerLink` followed by a path defined in the `app.routes.ts` file. You can route directly to a link or build path's that include dynamic values. For example, passing a `serviceId` to a different component, so we can display a single service.
 - e. Final thought: All components are classes and behave similar to JavaScript classes. That is, you can define variables and methods. In addition to the basic functionality of a class, Angular adds additional, built-in methods for state management (we will cover this in a later chapter).
5. Open a new terminal window and generate the following components: about, contact, services, and view-services.
- a. `ng generate component about -t -s`
 - b. `ng generate component contact -t -s`
 - c. `ng generate component services -t -s`
 - d. `ng generate component view-service -t -s`
6. Generate a new class.
- a. `ng generate class service`
7. Open the `service.ts` file and add the following code:

```

/**
 * Service interface to define the structure of a service item.
 */
export interface ServiceItem {
  id: number;
  name: string;
}

export class Service {
  services: Array<ServiceItem>; // array of ServiceItem objects

  constructor () {
    // array of ServiceItem objects
    this.services = [
      {
        id: 1,
        name: "Password Reset"
      },
      {
        id: 2,
        name: "Spyware Removal"
      },
      {
        id: 3,
        name: "RAM Upgrade"
      },
      {
        id: 4,
        name: "Software Installation"
      },
      {
        id: 5,
        name: "Tune-up"
      },
      {
        id: 6,
        name: "Keyboard Cleaning"
      }
    ]
  }
}

/**
 * @returns Returns an array of ServiceItem objects.

```

```

    */
    getServices(): Array<ServiceItem> {
        return this.services;
    }

    /**
     * @param id It will be used to find a service by id.
     * @returns If found, it will return a ServiceItem object otherwise it will throw
    an error message.
    */
    getService(id: number): ServiceItem {
        console.log('getService function called with id: ', id);
        console.log('Type of id: ', typeof id);

        // find method is used to find a service by id
        const s = this.services.find(service => service.id === Number(id));

        // if service is not found, throw an error message
        if (!s) {
            throw new Error(`Service ${id} not found`);
        }

        return s; // return the service
    }
}

```

In TypeScript, an interface is a way for you to define the structure of an object. It is a contract where you're saying, "Any object that uses this interface must include the properties and methods defined in the interface." Think of the interface as a list of rules. A class, on the other hand, allows you to create objects with shared behaviors and characteristics. JavaScript does not have a built-in concept for interfaces. TypeScript adds this feature to help developers define and enforce the structure of an object more strictly; however, it is important to note that interfaces are mainly used for type checking during development and are not included once the project is transpiled to native JavaScript. A class in TypeScript is very similar to a class in JavaScript (ES6 and beyond). The added benefit is, in TypeScript you can use interfaces within a TypeScript class to further enforce rules and to ensure your class must meet a specific criterion before its implemented correctly.

In the service.ts file, an interface (ServiceItem) and class (Service) are defined. An interface is used to ensure that the structure of a service item only contains an id and name property. In other words, if you use this interface, a service item can only ever have two properties: id and name. If you were to add a different property, say, price to the implementing object, I would throw an error. Likewise, if I were to assign

a string value to the id property, I would get an error. This is because of type checking.

Type checking in TypeScript is like having a gardening guide that tells you exactly what kind of seeds (data types) can be planted in which section of your garden (variables). Looking back at our previous gardening example, imagine you have a garden where you can plant flowers, vegetables, and fruits, but each type of plant needs its own specific area. If you accidentally plant a vegetable in the flower section, it might not grow well because the conditions are not appropriate for it. With type checking enabled, you would not be able to accidentally plant the vegetable in the flower section, because it would generate an error. It's a way to ensure you are following predefined rules when writing TypeScript code. In JavaScript type checking is not enforced, which means you can break rules during development without consequence (planting vegetables in the flower section). Errors are still thrown in JavaScript, but they are not caught until runtime (once an error is encountered in a web page; once the vegetable does not grow). The disadvantage of this approach is you may not know you have an error until it's too late (a user encounters it while using your application). With type checking, the errors are caught during development (before a user encounters it). Thus, as you can see, TypeScript is gaining a lot of traction and is extremely popular in the JavaScript community, especially on large codebases.

8. Open app.routes.ts and add the following Angular routes:

```
import { Routes } from '@angular/router';
import { ServicesComponent } from './services/services.component';
import { ViewServiceComponent } from './view-service/view-
service.component';
import { AboutComponent } from './about/about.component';
import { ContactComponent } from './contact/contact.component';

export const routes: Routes = [
  {
    path: 'about',
    component: AboutComponent
  },
  {
    path: 'contact',
    component: ContactComponent
  },
  {
    path: 'services',
    component: ServicesComponent
  },
```

```

{
  path: 'service/:id',
  component: ViewServiceComponent
}
];

```

In this code example, we define four routes. Each route is an object literal with two properties: path and component. The path property represents the path to the resource in your web application and the component property is the binding between the path and component. Meaning, “for this path, use this component.” Routes also allow route parameters. The `:id` part in `'service/:id'` is a placeholder for a dynamic value that will be passed through the URL. Routes are accessible by using their identifier. For example, the `'about'` path points to `/path` and the `'contact'` path points to `/contact`. With this in mind, we can access the `'service/:id'` path using `/service/123`. The `'123'` part will be captured as the value for the `id` parameter and is retrievable in the components class; however, before you can map a component to a path, you must import the component. We do this by using import statements that take the structure of **`import { [name] } from [package/module/component location]`**.

9. Open the `about.component.ts` file and add the following code:

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-about',
  standalone: true,
  imports: [],
  template: `
<div>
  <h1>About Us</h1>
  <p>Here is some information about us:</p>
  <ul>
    <li>Founded in 2001</li>
    <li>Located in Anytown, USA</li>
  </ul>
</div>
`,
  styles: ``
})
export class AboutComponent {

}

```

10. Open the contact.component.ts file and add the following code:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-contact',
  standalone: true,
  imports: [],
  template: `
    <div>
      <h1>Contact Us</h1>
      <p>Here is our contact information:</p>
      <ul>
        <li>Phone: 555-555-5555</li>
        <li>Email: support@me.com</li>
      </ul>
    </div>
  `,
  styles: ``
})
export class ContactComponent {

}
```

11. Open the service.component.ts file and add the following code:

```
import { Component } from '@angular/core';
import { Service, ServiceItem } from '../service';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-services',
  standalone: true,
  imports: [RouterLink],
  template: `
    <div>
      <h1>Our Services</h1>
      <p>Here are the services we offer:</p>
      <ul>
        @for(serviceltem of services; track serviceltem) {
          <li> <a routerLink="/service/{{serviceltem.id}}"> {{ serviceltem.name
        }}</a></li>
        }
      </ul>
    </div>
  `
})
export class ServiceComponent {

}
```

```

</div>
`,
styles: ``
})
export class ServicesComponent {
  services: Array<ServiceItem>;

  constructor() {
    const service = new Service();
    this.services = service.getServices();
  }
}

```

While this component is relatively small, there is a lot going on. This code defines an Angular component named `ServicesComponent`, which is responsible for displaying a list of services. Here is a breakdown of the key properties in this component:

- a. **selector:** Specifies the custom HTML tag `<app-services>` that will be used to call this component. You can also use this custom HTML tag in other components in your application and each time Angular encounters this HTML tag, it will invoke the components code (HTML, CSS, and Class code).
- b. **standalone:** Indicates that the component is standalone, meaning it can be used without being declared as a module (covered in the last chapter of this book).
- c. **imports:** List the external components or directives used in this component. In our case, we are using `RouterLink` from Angular's routing library to enable navigation.
- d. **template:** This section is for the components view (HTML code) and is similar to how traditional HTML development with the added benefit of having access to Angular's templating syntax (`@for`).
- e. **styles:** This section is for the components CSS styling (similar to how you would normally style a static webpage using CSS).

As of Angular 17, a new template syntax was introduced to conditionally render elements in Angular projects. This new template syntax is only available when using inline HTML templates. If you are using a separate HTML file to generate the components view, the new template syntax is unavailable and will generate compile-time exceptions. Here are some of the most commonly used items:

- a. **@if:** This is a block conditional and is used to test the truthy of an expression. For example, `@if (name === 'Professor Krasso') { // HTML code goes here }`. You can also chain `if...else` statements together to form complex conditionals.

```
@if (grade === 'A') {  
  // HTML code  
} @else if (grade === 'B') {  
  // HTML code  
} @else if (grade === 'C') {  
  // HTML code  
} @else if (grade === 'D') {  
  // HTML code  
} @else {  
  // HTML code  
}
```

- b. **@for:** Similar to a “for loop,” this is used to “loop over” the content in an array or data collection.

```
@for(grade of grades; track grade) {  
  {{grade}}  
}
```

The value of `track` “determines a key used to associate array items with the views in the DOM (Angular, 2024).” Having a “clear indication of the item identity allows Angular to execute a minimal set of DOM operations as items are added, removed, or moved in a collection (Angular, 2024).”

- c. **@empty:** This block is used to optionally execute a block of code after a `@for` block has finished executing. This can be useful when you want to display a custom message for empty data collections. For example, let’s say we are calling an API that returns a list of users. We could use the `@empty` block to display a custom message of “No users found” if the API returns an empty collection of user objects.

```
@for (user of users; track user) {  
  <li> {{ user.name }} </li>  
} @empty {  
  <p> There are no users. Consider adding one. </p>  
}
```

- d. **@switch:** This block is similar to switch statement in traditional programming languages. It is inspired by JavaScript's switch statement and allows you to conditionally check the truthy of a value.

```
@switch (grade) {  
  @case (A) {  
    // HTML code  
  }  
  @case (B) {  
    // HTML code  
  }  
  @default {  
    // HTML code  
  }  
}
```

- e. **@let:** This syntax allows you to define a local variable to be used throughout the HTML code.

```
@let name = "Professor Krasso"  
@let title = "Associate Professor"  
  
<div> Name: {{ name }} </div>  
<div> Title: {{ title }} </div>
```

The code in the body of the `ServicesComponent` class defines a variable named `services` of type `Array<ServiceItem>` and includes a constructor. In the body of the constructor, a variable is declared and initialized with an instance of the `Service` class and the `services` variable is assigned the response from calling the `getServices()` method. In the HTML code, a for loop is used to iterate over the `services` array and `<li>`'s are used to display the available services. Each `<li>` includes an anchor tag with a `routerLink` that points to the `/services/:id` path defined in our `app.routes.ts` file.

12. Open the `view-service.component.ts` file and add the following code:

```
import { Component } from '@angular/core';  
import { Service, ServiceItem } from '../service';  
import { ActivatedRoute, RouterLink } from '@angular/router';  
  
@Component({  
  selector: 'app-view-service',  
  standalone: true,  
  imports: [RouterLink],
```

```

template: `
<div>
  <h1>Service Details</h1>
  <p>Here are the details of the service:</p>
  <ul>
    <li>Service ID: {{ serviceltem.id }}</li>
    <li>Service Name: {{ serviceltem.name }}</li>
  </ul>

  <a routerLink="/services">Back to services</a>
</div>
`,
styles: ``
})
export class ViewServiceComponent {
  serviceltem: Serviceltem;
  serviceId: number;

  constructor(private route: ActivatedRoute) {
    this.serviceId = this.route.snapshot.params['id'];
    console.log(this.serviceId);

    const service = new Service();
    this.serviceltem = service.getService(this.serviceId);
  }
}

```

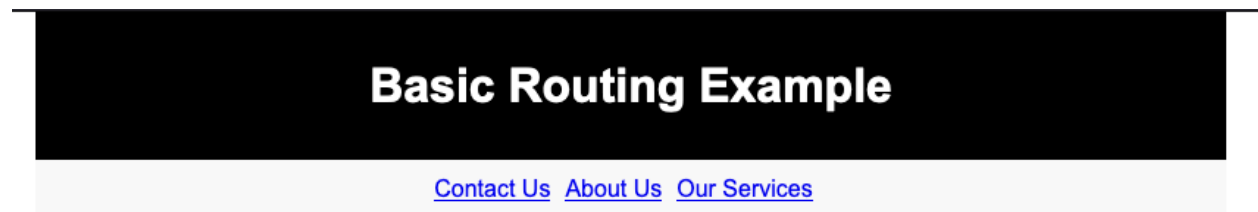
Given we already covered some of this codes functionality in the previous component (services.componet.ts), I will only focus on the new parts:

- a. **import { ActivatedRoute } from '@angular/router'**: This import statement gives us access to the ActivatedRoute library and methods for extracting data from router parameters.
- b. **constructor(private route: ActivatedRoute) { ... }**: This is using dependency injection. Imagine you are planting a garden and have a variety of plants that need different types of care – water, sunlight, soil nutrients, etc., Instead of having to remember and provide each plant with its specific needs, you have a smart gardening system. When you plant something, you tell the system what you've planted, and it automatically takes care of providing the right amount of water, sunlight, and nutrients. Dependency injection is like that. It's telling the smart gardening system, "Here is a new plant (the component), that needs this type of care (the ActivatedRoute)" and the smart gardening

system (dependency injection) takes care of the steps needed to provide everything you need to use the library.

- c. **this.route.snapshot.params['id']**: This line of code uses the `ActivatedRoute` variable from the constructor (see step b) and calls the `.snapshot` object's `.params` property to access the placeholder `:id` from the path `/services/:id` in the `app.routes.ts` file. Effectively, giving us access to the services id value.

Let's run and test the application to confirm there are no errors in the console and we can view each of the components. Open a new terminal window and start the server (ng serve). If you followed the tutorial correctly you should see a landing page three menu items:



Selecting the “Contact Us” link should display:

Contact Us

Here is our contact information:

- Phone: 555-555-5555
- Email: support@me.com

Selecting the “About Us” link should display:

About Us

Here is some information about us:

- Founded in 2001
- Located in Anytown, USA

Selecting the “Our Services” link should display:

Our Services

Here are the services we offer:

- [Password Reset](#)
- [Spyware Removal](#)
- [RAM Upgrade](#)
- [Software Installation](#)
- [Tune-up](#)
- [Keyboard Cleaning](#)

And, selecting a service should display:

Service Details

Here are the details of the service:

- Service ID: 1
- Service Name: Password Reset

[Back to services](#)

If you have errors in the terminal window or the components are not displaying as shown above, it is likely you have a typo or syntax error in your code.

In the next section, we will continue where we left off in the virtual-taco-stand project by adding the components we will need for the remaining chapters in this book.

Routing in Action

In this section we will be generating the components and defining the routes we will need for the remaining chapters in this book for our virtual-taco-stand application.

Continuing where we left off in the virtual-taco-stand project:

1. Open the virtual-taco-stand project in your editor.
2. Generate components for daily-specials, feedback, menu, order, and signin.

- a. ng generate component daily-specials -t -s
 - b. ng generate component feedback -t -s
 - c. ng generate component menu -t -s
 - d. ng generate component order -t -s
 - e. ng generate component signin -t -s
3. Define the routes for each component in the app.routes.ts file.

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { MenuComponent } from './menu/menu.component';
import { OrderComponent } from './order/order.component';
import { DailySpecialsComponent } from './daily-specials/daily-specials.component';
import { FeedbackComponent } from './feedback/feedback.component';
import { SigninComponent } from './signin/signin.component';
```

```
export const routes: Routes = [
  {
    path: '',
    component: HomeComponent
  },
  {
    path: 'home',
    component: HomeComponent
  },
  {
    path: 'menu',
    component: MenuComponent
  },
  {
    path: 'order',
    component: OrderComponent
  },
  {
    path: 'daily-specials',
    component: DailySpecialsComponent
  },
  {
```

```

    path: 'feedback',
    component: FeedbackComponent
  },
  {
    path: 'signin',
    component: SigninComponent
  }
];

```

4. Update the links in the app.component.ts file to the paths we defined in the app.routes.ts file.

```

import { Component } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="wrapper">
      <header class="banner">
        
      </header>

      <div class="sign-in-container">
        <a routerLink="/signin" class="sign-in-link">Sign In</a>
      </div>

      <main class="main-content">
        <nav class="navbar">
          <ul>
            <li><a routerLink="/">Home</a></li>
            <li><a routerLink="/menu">Menu</a></li>
            <li><a routerLink="/order">Order</a></li>
            <li><a routerLink="/daily-specials">Daily Specials</a></li>
            <li><a routerLink="/feedback">Feedback</a></li>
          </ul>
        </nav>
        <section class="content">
          <router-outlet />
        </section>
      </main>

```

```

<footer class="footer">
  <nav class="footer-nav">
    <a routerLink="/">Home</a> |
    <a routerLink="/menu">Menu</a> |
    <a routerLink="/order">Order</a> |
    <a routerLink="/daily-specials">Daily Specials</a> |
    <a routerLink="/feedback">Feedback</a>
  </nav>
  <p>&copy; 2024 Virtual Taco Stand</p>
</footer>
</div>
`,
styles: [
  `
    .sign-in-container {
      text-align: right;
      padding-right: 20px;
      margin-top: 10px;
    }

    .sign-in-link {
      color: #000000;
      text-decoration: none;
      font-family: 'Lato', sans-serif;
    }

    .sign-in-link:hover {
      text-decoration: underline;
    }
  `,
]
})
export class AppComponent {
}

```

Run and test the project (ng serve) to ensure there are no errors in the terminal window and navigation is working between the routes we defined in the app.routes.ts file (the side menu and footer menu should be fully functional).

Hands-On 2.1: Routing in Action

In this hands-on project, you will be extending the rpg-character-builder application by adding the components and routing you will need for the remaining hands-on projects in this book.

Instructions:

1. Continue working in the project from the previous chapter's hands-on project.
2. You will be generating the following components in your project (use inline templates and inline CSS): players, signin, create-character, create-guild, and character-faction. In your project, ensure you have the following file structure:

```
public
src
  app
    index.html
    main.ts
    styles.css
    app.component.ts
    app.component.spec.ts
    app.config.ts
    app.routes.ts
  players
    players.component.ts
    players.component.spec.ts
  signin
    signin.component.ts
    signin.component.spec.ts
  create-character
    create-character.component.ts
    create-character.component.spec.ts
  create-guild
    create-guild.component.ts
    create-guild.component.spec.ts
  character-faction
    character-faction.component.ts
    character-faction.component.spec.ts
```

3. Define routes for each of the components generated in step 2 in the app.routes.ts file.

4. Update the `app.component.ts` with navigational links for each of the components that were generated and defined in steps 2 and 3.

Grading:

You will earn 20 points for each step (2, 3, and 4), for a total of 60 points. If one step is omitted, you will earn 40 points.

Hints:

- Using `ng generate component [component name] -t -s` to generate a new component.
- Use `{ path: '/path', component: ComponentName }` in the `app.routes.ts` file to define your routes.
- Use `<a routerLink='/path'>Link Name</a>` in the `app.component.ts` file to register your links.
- Don't forget to import the `RouterLink` in the `app.component.ts` file.

Chapter 3. Error Handling and Unit Testing

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Discover error handling techniques in Angular.
- Measure the effectiveness of unit testing in Angular.
- Justify the use of Test-Driven Development (TDD) in Angular projects.
- Recommend configurations for Karma testing in Angular.

Error Handling

Proper error handling is key to the longevity and success of any web application. Error handling best practices are language agnostic and are intended to aid developers in ensuring their code is free of errors (logical, syntax, etc.) and fault tolerate (protected against potential errors). There are several built-in mechanisms within Angular that make Angular projects easy to debug and troubleshoot. In this section we will cover some of the most important aspects of proper error handling in an Angular project.

Here are the top strategies for error handling in an Angular project:

1. **Global Error Handling:** In this approach, one implements global error handling by extending the `ErrorHandler` class. This type of error handling is designed to catch errors that are not caught by other error handling strategies implemented in your Angular application.

2. **HTTP Interceptors (not covered in this book):** HTTP Interceptors, also known as route interceptors, are a strategy for working with HTTP request errors. Route interceptors can be used to catch HTTP request and response errors globally. In addition, route interceptors can be used to log incoming and outgoing HTTP transactions.
3. **Try-Catch for Promises and Async/Await:** Use try-catch blocks to handle errors from asynchronous operations, like promises and async/await. This approach provides a streamlined strategy for mitigating asynchronous failures, gracefully.
4. **User-Friendly Error Messages:** Translate technical errors into human-readable and friendly text that is easy to understand and actionable. Actionable, in the context of error handling, means messages should be clear, informative, and usable.
5. **Error Logging:** Implementing an error logging strategy is a great way to keep track of the interactivity in your application. This entails sending errors to a backend service or API that stores the information in a text file or database system. Thus, enabling you to make informed decisions on how to resolve issues impacting user experiences.
6. **Retry Strategies:** For network errors, it is typically recommended to introduce some type of retry strategy to programmatically re-execute failures. This can improve the users experience by reducing the chances a failure will occur due to some type of temporary issue or network connectivity problem.

In addition to the top strategies for error handling, one must be aware of versioning and compatibility issues that are common in Angular projects. In Angular, itself, there are many third-party libraries and dependencies. These dependencies and libraries have ongoing releases and updates that may impact the functionality of your Angular application. Here are some strategies that are useful in handling these types of issues:

1. **Keep Angular Updated:** Use the [Angular Update Guide](#) for step-by-step instructions on how to update an Angular application and its dependencies to the latest version.
2. **Semantic Versioning:** Learn semantic versioning (major.minor.patch) to anticipate the impact of updating a dependency in your project. Major versions typically introduce breaking changes and could have negative impacts on your project. Carefully test major version changes in a separate git branch to explore the potential impacts it might have on your project.
3. **Angular CLI:** Angular CLI helps manage dependences and automates many tasks. Using commands like **ng update** will check your project for version compatibility issues and apply migrations.

4. **Testing:** After updating your project to a new version, always rerun unit tests and thoroughly test the application for potential compatibility issues.
5. **Third-Party Libraries:** If you are using third-party libraries, check the libraries documentation or GitHub issues for discussions related to compatibility and breaking changes.
6. **Community and Support:** Angular's community is vast with plenty of resources available to help you troubleshoot versioning issues. Resources like Angular forums, Stack Overflow, and GitHub will offer solutions to commonly experienced problems during versioning updates.

Let's take a look at a simple Angular project that demonstrates some of the items outlined in this section. As a disclaimer, the examples in this project are for demonstration purposes only and should not be considered "production ready." The project is named error-handling-demo and it can be found in the data-files folder.

1. Open the error-handling-demo project in your code editor.
2. **Global Error Handling:** Open the file named global-error-handler.service.ts. In this file, the GlobalErrorHandlerService class is implementing the ErrorHandler interface provided by Angular. In TypeScript, the "implements" keyword is used to ensure that a class conforms to a particular interface. In other words, by implementing the ErrorHandler interface, the GlobalErrorHandlerService must provide an implementation for the properties and methods defined in the ErrorHandler interface. And, in our case, it's a single method called errorHandler(). The handleError() method takes a formal parameter of type "any." In the context of this method, the type "any" is used to allow the method to handle different kinds of errors that might occur in your web application. In the body of this method, we have a single console.error() call with a string message indicating an error occurred that was caught by the global error handler.
3. Open the file named app.config.ts. The app.config.ts file is a configuration file that Angular uses to assemble your application. It serves as a centralized location for application-wide settings and services. This includes global routing, and in our case, global error handling, which is registered by calling

{ provide: ErrorHandler, useClass: GlobalErrorHandlerService }

4. **Try-Catch for Promises and Async/Await:** Open the component-level-error-handling.component.ts file. This component illustrates how you can programmatically handle component level errors with a try-catch block and string assignment.

5. **Error Logging:** Open the `logger.service.ts` file. This is a TypeScript service with three methods: `log()`, `error()`, and `warn()`. Each of these methods call a different method from the console object.
6. Open the `component-logger-error-handling.ts` file. This component illustrates how you can use the `logger.service.ts` file to implement logging in your Angular application.

To test the application, run `ng serve` from a new CLI window. Each of the links in the landing page have instructions on how to view the error handling messages. The key takeaway is there are a variety of strategies that you can leverage in your Angular application for responding and handling errors in an efficient manner.

In the next section, we will take a look at how to write unit tests in an Angular project.

Unit Testing

In this section, we will explore TDD principles and how to apply them to an Angular project. Out of the box, Angular uses Karma as its default test runner for unit tests. Karma is a test runner developed by the Angular team that executes your test suites and test cases, providing an environment where you can run tests across multiple browsers. It is not a test framework, instead it was designed to integrate with other test frameworks. Angular's default test framework is Jasmine; however, support has been provided for other test frameworks like, Mocha, QUnit, Jest (test framework and test runner), etc., The main advantage of Angular's test runner is its ability to simulate browser environments. To keep the examples in this book simple, we will be using Angular's default test runner (Karma) and test framework (Jasmine) to build our unit tests. Fortunately, if you've worked with other test frameworks, learning how to use Karma and Jasmine should be straightforward.

By default, each time you generate a new file using Angular's CLI, a test file with a `.spec.ts` is auto-generated. This test file is intended to be used as the test suite for the file you generated. For example, let's say I created a new component called "home." Assuming I used Angular's CLI to generate this component, a new file will be added to the project called `home.component.spec.ts`. All of the unit tests you build to test the `HomeComponent` should be placed in this file. To run unit tests, Angular includes a script in the `package.json` file called `test`, which is executable by running the CLI command **ng test**. Here are some of the most commonly used test methods in Jasmine:

1. **describe(string, function):** Defines a test suite, which is a collection of unit tests.
2. **it(string, function):** Defines a single unit test.
3. **beforeEach(function):** Setup method that runs before each unit test in a test suite.

4. **afterEach(function):** Teardown method that runs after each unit test is executed in the test suite. Typically, the `afterEach()` method is used for releasing resources and “clean-up” work.
5. **beforeAll(function):** Similar to `beforeEach()`, but instead of running before each unit test in the test suite, it runs once before all unit tests in the test suite are executed.
6. **afterAll(function):** Similar to `afterEach()`, but instead of running after each unit test in the test suite is executed, it runs once after all unit tests in the suite are executed.
7. **expect(value):** Used to create an exception for a test, which is used in combination with another test method to test some type of criteria.
8. **toBe(value):** A matcher method that tests if the expected value matches some value. Used in combination with `expect()` method. For example, `expect(value).toBe(value)`.
9. **toEqual(value):** A matcher that checks if the expected value matches some value. The difference between the `toBe()` method and `toEqual()` method is how the equality is compared. In `toBe()` the value is strictly compared (`===`), but in `toEqual()` the comparison can perform deep equality checks for objects (all properties in two objects strictly match). Used in combination with the `expect()` method. For example, `expect(value).toEqual(value)`.
10. **toHaveBeenCalled():** A matcher method that verifies whether a spy function has been called.
11. **toHaveBeenCalledWith(arg1, arg2, ...):** Similar to the `toHaveBeenCalled()` method, but with specific actual parameters.
12. **toBeTruthy():** A matcher method that checks if the value in the `expect()` method is true. For example, `expect(value).toBeTruthy()`.
13. **toBeFalsy():** Similar to the `toBeTruthy()` method, but it checks the `expect()` method value to be false. For example, `expect(value).toBeFalsy()`.
14. **toContain(item):** A matcher method that checks whether an array or string contains the actual parameter item.
15. **toBeGreaterThan(number), toBeLessThan(number):** matchers that check the `expect()` method value is either greater than or less than the given number.

Now that you have a general idea of the most commonly used test methods, let's take a look at a simple Angular project that leverages some of these methods. The project used in the next section is named unit-testing-demo and it is located in the data-files folder.

1. Open the unit-testing-demo in your code editor.
2. Open the karma.config.js file. This is a configuration file for the Karma test runner and it is used to provide instructions to Karma for setting up and configuring your unit tests. In other words, it defines how unit tests are executed in your Angular project. Of note, is the browsers property, which specifies which browser Karma will launch when it executes the applications unit tests. As you can see from the project, I am using "Firefox" to execute the unit tests. By default, a new Angular project is preconfigured to use "Chrome." But you can change this value to whatever your computer's default browser is. Just understand, whatever browser you set in this section must be the installed on your computer. In other words, if you do not have "Firefox" installed on your computer, the unit tests will fail. Here are the supported browser options:
 - a. **Chrome:** Requires the npm package karma-chrome-launcher
 - b. **ChromeCanary:** Requires the npm package karma-chorme-launcher
 - c. **ChromeHeadless:** Requires the npm package karma-chrome-launcher
 - d. **PhantomJS:** Requires the npm package karma-phantomjs-launcher
 - e. **Firefox:** Requires the npm package karma-firefox-launcher
 - f. **Opera:** Requires the npm package karma-opera-launcher
 - g. **IE:** Requires the npm package karma-ie-launcher
 - h. **Safari:** Requires the npm package karma-safari-launcher

Plugins must be registered in the karma.conf.js file under the plugins array. And, supported browsers must be added to the browsers array.

// Karma configuration file, see link for more information
// <https://karma-runner.github.io/1.0/config/configuration-file.html>

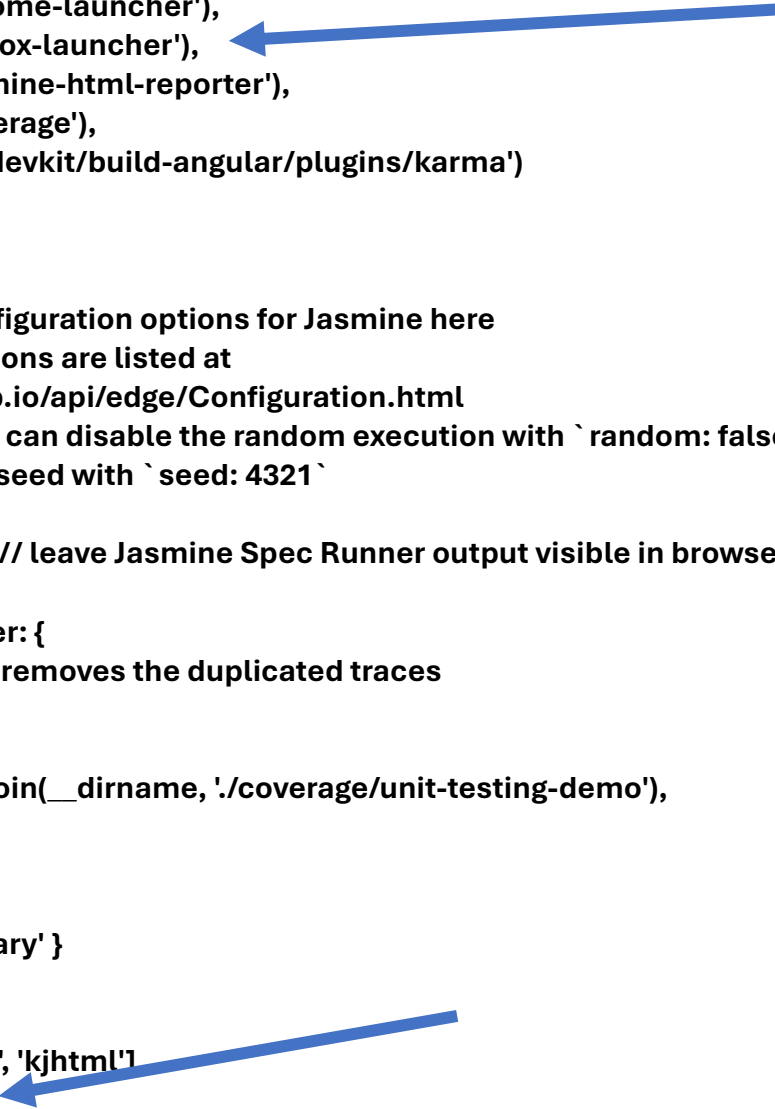
```
module.exports = function (config) {  
  config.set({  
    basePath: "",  
    files: [  

```

```

    { pattern: 'src/**/*.spec.ts', type: 'js' }
  ],
  frameworks: ['jasmine', '@angular-devkit/build-angular'],
  plugins: [
    require('karma-jasmine'),
    require('karma-chrome-launcher'),
    require('karma-firefox-launcher'),
    require('karma-jasmine-html-reporter'),
    require('karma-coverage'),
    require('@angular-devkit/build-angular/plugins/karma')
  ],
  client: {
    jasmine: {
      // you can add configuration options for Jasmine here
      // the possible options are listed at
https://jasmine.github.io/api/edge/Configuration.html
      // for example, you can disable the random execution with `random: false`
      // or set a specific seed with `seed: 4321`
    },
    clearContext: false // leave Jasmine Spec Runner output visible in browser
  },
  jasmineHtmlReporter: {
    suppressAll: true // removes the duplicated traces
  },
  coverageReporter: {
    dir: require('path').join(__dirname, './coverage/unit-testing-demo'),
    subdir: '.',
    reporters: [
      { type: 'html' },
      { type: 'text-summary' }
    ]
  },
  reporters: ['progress', 'kjhtml'],
  browsers: ['Firefox'],
  restartOnFileChange: true
});
};

```



You can generate the karma.config.js file by using the following CLI command:

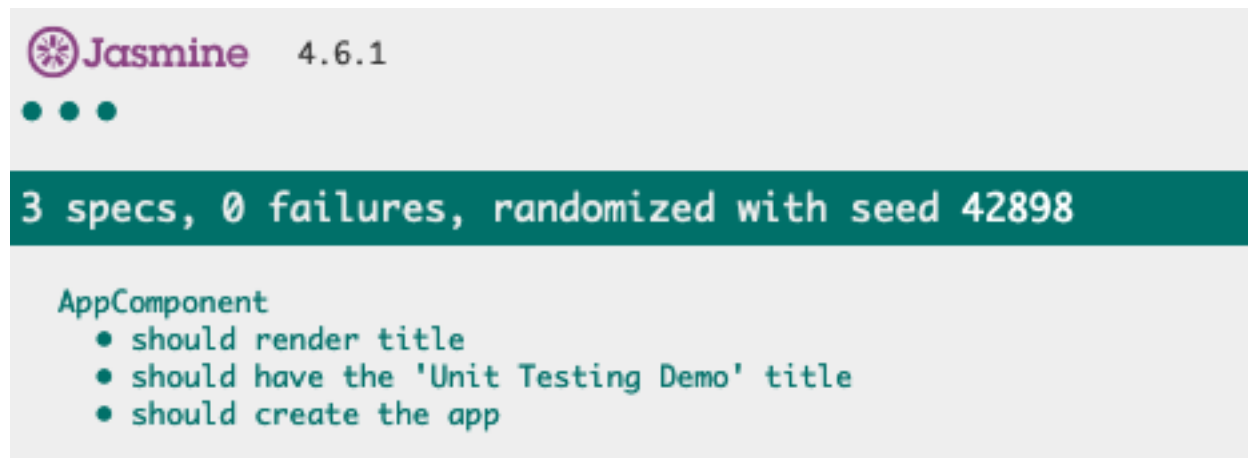
ng generate config karma

And, to install the Firefox plug-in, use the following command:

npm install karma-firefox-launcher --save-dev

The projects in this book have been tested using Firefox and Chrome. If you want to use Firefox (unit-testing-demo has already been configured for Firefox), you will need to configure the starter projects (located in the data-files directory) to use it by following the steps outlined in this section (generate a karma configuration file, install the plug-in, and update the karma file with the appropriate settings). For Chrome, no additional configuration is needed as it is already supported by default. If you prefer to use a different browser, you will need to configure the project accordingly. Given the complexities associated with configuring the Karma test runner, I highly recommend you stick with Chrome or Firefox.

3. To run our unit tests, open a new CLI window and enter `ng test`. A new browser window should open with the results of the unit tests. To ensure the unit tests run, you must set the value in the browsers (karma.conf.js) array to either Chrome or Firefox, depending on which browser you have installed on your computer. If you prefer to use a different browser, follow the steps outlined above.



4. Open the `app.component.spec.ts` file and add the following unit tests:

```
// Simple addition test  
it('should add two numbers correctly', () => {  
  const a = 5;  
  const b = 3;  
  const result = a + b;  
  expect(result).toBe(8);  
});
```

```
// Simple subtraction test
```

```

it('should subtract two numbers correctly', () => {
  const a = 10;
  const b = 4;
  const result = a - b;
  expect(result).toEqual(6);
});

// Check if a function has been called (using a spy)
it('should check if a method has been called', () => {
  const spyObj = jasmine.createSpyObj('AppComponent', ['dummyMethod']);
  spyObj.dummyMethod();
  expect(spyObj.dummyMethod).toHaveBeenCalled();
});

// Check if a value is truthy
it('should check if a value is truthy', () => {
  const isAvailable = true;
  expect(isAvailable).toBeTruthy();
});

```

Each of these unit tests demonstrate how we can leverage some of Jasmines core methods to write unit tests. In the next section, we will take a look at how to apply TDD principles to Angular components.

Angular Template Syntax

In this section we will explore how to apply TDD principles to the development of a MenuComponent in the virtual-taco-stand project. The tutorial presented in this section assumes you are using Chrome as the default browser for the Karma test runner. If you would like to use a different browser (not recommended), then you must first configure it, before proceeding with the steps listed below:

Continuing with where we left off in the virtual-taco-stand project:

1. Open the project in your code editor.
2. Open the app.component.spec.ts file and update it with the following:

```

import { TestBed } from '@angular/core/testing';
import { AppComponent } from './app.component';
import { ActivatedRoute } from '@angular/router';
import { of } from 'rxjs';
import { RouterTestingModule } from '@angular/router/testing';
import { MenuComponent } from './menu/menu.component';

```

```

import { Routes, Router } from '@angular/router';

describe('AppComponent (Standalone)', () => {
  beforeEach(async () => {
    const activatedRouteStub = {
      snapshot: {
        paramMap: {
          get: () => 'staticValue',
        },
      },
      queryParams: of({}),
    };

    const routes: Routes = [
      { path: 'menu', component: MenuComponent }
    ]

    await TestBed.configureTestingModule({
      imports: [
        RouterTestingModule.withRoutes(routes), // Include RouterTestingModule
        MenuComponent
      ],
      providers: [
        { provide: ActivatedRoute, useValue: activatedRouteStub },
      ],
    }).compileComponents();
  });

  it('should create the app', () => {
    const fixture = TestBed.createComponent(AppComponent); // Directly create
    the component
    const app = fixture.componentInstance;
    expect(app).toBeTruthy();
  });

  /** Week 3: unit test 3 */
  it('should have correct route for Menu Component', () => {
    const router = TestBed.inject(Router);
    const route = router.config.find(r => r.path === 'menu');
    expect(route).toBeDefined(); // Check if the route is defined
    if (route) {
      expect(route.component).toBe(MenuComponent); // Check if the
      component is MenuComponent
    }
  });
});

```

```

    }
  });
});

```

In this code example, we've updated the unit tests `app.component.spec.ts` file to verify the functionality of our `AppComponent`. Let's dive into each part of the code to understand how it works.

- **Import statements:** `ActivatedRoute`, `of`, `RouteTestingModule`, `Routes`, and `Router` are needed to test routing in our component (see the next section).
 - **const activatedRouteStub:** This block of code defines a stub for `ActivatedRoute`. It mimics the structure of the `ActivatedRoute` module used in Angular. We need this for the `snapshot.paramMap.get` method. In other words, this code is used to mimic route parameters and query parameters in a unit test without having to directly reference the actual router.
 - **const routes: Routes:** This block defines an array of routes for the application. This is part of the test suites configuration settings and is needed for testing components with routing.
 - **await TestBed.configureTestingModule:** This block of code is setting up the testing environment for the components being tested. The imports section, imports the necessary modules for the test. Next, the providers array "provides" the dependencies for the component that is being tested. And, finally, the `compileComponents()` method is called to compile the components involved in the test suite.
 - **it('should create the app', () => { ... }):** This unit test is testing if the `AppComponent` is being created. By default, all newly generated components automatically have this unit test.
 - **It('should have correct route for MenuComponent', () => { ... }):** This unit test is testing if the link in the `app.component.ts` file for the `MenuComponent` is working.
3. Open the `menu.component.spec.ts` file and add a unit test that checks if `MenuComponent` correctly displays a list of tacos:

```

import { ComponentFixture, TestBed } from '@angular/core/testing';

import { MenuComponent } from './menu.component';

describe('MenuComponent', () => {

```

```

let component: MenuComponent;
let fixture: ComponentFixture<MenuComponent>;

beforeEach(async () => {
  await TestBed.configureTestingModule({
    imports: [MenuComponent]
  })
  .compileComponents();

  fixture = TestBed.createComponent(MenuComponent);
  component = fixture.componentInstance;
  fixture.detectChanges();
});

/**
 * Created by default when the component is generated
 * Unit test 1
 */
it('should create', () => {
  expect(component).toBeTruthy();
});

/**
 * Unit test 2
 */
it('should correctly display a list of tacos', () => {
  const compiled = fixture.nativeElement as HTMLElement; // Get the compiled
HTML of the component
  const menuItems = compiled.querySelectorAll('.menu-item'); // Get all the
menu items
  expect(menuItems.length).toEqual(component.menu.length); // Check if the
number of menu items is equal to the number of items in the menu array
});
});

```

In this code example, we've added a new unit test that checks if the MenuComponent displays a list of tacos. Let's take a closer look at what this unit test is doing.

- **const compiled = fixture.nativeElement as HTMLElement:** This line of code is getting the compiled HTML code from the MenuComponent. We need this so we check the DOM for a list of tacos.
- **const menuItems = compiled.querySelectorAll('.menu-item');** This line of code is getting all of the nested menu items from the .menu-item CSS selector.

- **expect(menuItems.length).toEqual(component.menu.length):** This line of code checks if the number of menu items is equal to the number of items in the menu array. Menu items are from the .menu-item CSS selector and component.menu.length is from the variable defined in the menu.component.ts files MenuComponent class.

To ensure our tests fail, run ng test from a new CLI window.

Passing Test (Green):

1. Open the menu.component.ts file.
2. Write the HTML, CSS, and TypeScript code to display a list of tacos, representing the taco stands menu.

```
export interface MenuItem {
  name: string;
  description: string;
  price: number;
}
```

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
```

```
@Component({
  selector: 'app-menu',
  standalone: true,
  imports: [ CommonModule ],
  template: `
    <div>
      <h1>Our Menu</h1>
      <p>Explore our selection of handcrafted tacos, each filled with fresh
ingredients and vibrant flavors to satisfy your cravings.</p>
```

```
    <ul class="menu-container">
      @for (item of menu; track item) {
        <li class="menu-item">
          <div class="card">
            <h3>{{ item.name }}</h3>
            <p>{{ item.description }}</p>
            <p>{{ item.price | currency }}</p>
          </div>
        </li>
      }
    </ul>
```

```

    }
  </ul>
</div>
`,
styles: [

  .menu-container {
    display: flex;
    flex-wrap: wrap;
    list-style-type: none;
    padding: 0;
  }

  .menu-item {
    flex: 0 1 calc(33.33% - 20px);
    margin: 10px;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  }

  .card {
    padding: 20px;
    background-color: white;
  }
,
]
})
export class MenuComponent {
  menu: MenuItem[];

  constructor() {
    this.menu = [
      {
        "name": "Carnitas Taco",
        "description": "Slow-cooked pork with fresh cilantro, onions, and salsa on a corn tortilla.",
        "price": 3.25
      },
      {
        "name": "Queso Birria Taco",
        "description": "Cheesy birria with cilantro, onions, and consomé for dipping.",
        "price": 3.50
      },
      {

```

```

    "name": "Al Pastor Taco",
    "description": "Marinated pork with pineapple, cilantro, and onions on a
corn tortilla.",
    "price": 3.25
  },
  {
    "name": "Tacos de Lengua",
    "description": "Tender beef tongue with cilantro and onions on a corn
tortilla.",
    "price": 3.50
  },
  {
    "name": "Chicken Taco",
    "description": "Grilled chicken with lettuce, tomatoes, and salsa on a corn
tortilla.",
    "price": 3.25
  },
  {
    "name": "Fish Taco",
    "description": "Battered fish with cabbage slaw and creamy sauce on a flour
tortilla.",
    "price": 3.25
  },
  {
    "name": "Veggie Taco",
    "description": "Grilled vegetables with black beans, cheese, and salsa on a
corn tortilla.",
    "price": 3.25
  },
  {
    "name": "Chicharron Taco",
    "description": "Crispy pork rinds with salsa on a corn tortilla.",
    "price": 3.25
  },
  {
    "name": "Potato Taco",
    "description": "Fried potatoes with lettuce, cheese, and salsa on a corn
tortilla.",
    "price": 3.25
  },
  {
    "name": "Chorizo Taco",
    "description": "Spicy sausage with onions and cilantro on a corn tortilla.",
    "price": 3.25
  }

```

```

    },
  ];
}
}

```

In this code example, we write the HTML, CSS, and TypeScript code for displaying a list of tacos available on our menu. Let's take a closer look at the functionality of this component:

- **@for(item of menu; track item) { ... }**: This block of code is using Angular's HTML template syntax to iterate over the menu array and output each menu item in a separate CSS card.
- **export interface MenuItem { ... }**: This block of code defines a TypeScript interface named MenuItem with properties for name, description, and string. We use this interface in the MenuComponent class to build an array of MenuItem objects.
- **menu: MenuItem[] and this.menu = []**: These two blocks of code declare a variable named menu of type MenuItem array and assign it a list of menu item objects.

Rerun the unit tests to ensure they are passing (ng test). Assuming you have no errors in your codebase, all unit tests should pass. Next, start the server to verify the "Menu" link is working (left-hand navigation menu) and the MenuComponent displays the taco stands menu. There should be a total of 10 menu items, displayed in rows of 3. If you do not see the menu or there is an error in the console, please go back through this section again to ensure the code you've added matches what I've presented. Usually, if there is an error, it can be directly attributed to syntax, misspelled words, and missing closing tags/elements.

Menu Display:

Our Menu

Explore our selection of handcrafted tacos, each filled with fresh ingredients and vibrant flavors to satisfy your cravings.

Carnitas Taco Slow-cooked pork with fresh cilantro, onions, and salsa on a corn tortilla. \$3.25	Queso Birria Taco Cheesy birria with cilantro, onions, and consomé for dipping. \$3.50	Al Pastor Taco Marinated pork with pineapple, cilantro, and onions on a corn tortilla. \$3.25
Tacos de Lengua Tender beef tongue with cilantro and onions on a corn tortilla. \$3.50	Chicken Taco Grilled chicken with lettuce, tomatoes, and salsa on a corn tortilla. \$3.25	Fish Taco Battered fish with cabbage slaw and creamy sauce on a flour tortilla. \$3.25

Passing Tests:

9 specs, 0 failures, randomized with seed 59695

```
SigninComponent
  • should create

DailySpecialsComponent
  • should create

AppComponent (Standalone)
  • should have correct route for Menu Component
  • should create the app

HomeComponent
  • should create

FeedbackComponent
  • should create

MenuComponent
  • should correctly display a list of tacos
  • should create

OrderComponent
  • should create
```

Hands-On 3.1: Unit Testing

In this hands-on project, you will be extending the rpg-character-builder by building the “Players” page to display a list of preconfigured characters. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter’s hands-on project.
2. Write the HTML, CSS, and TypeScript code for the PlayersComponent. At minimum, you must include the following:
 - a. Display a list of 10 characters stored in an array of character objects. Include the following properties: name, gender (male, female, other), class (Warrior, Mage, Rogue), faction, startingLocation, funFact. For example,

```
"name": "Thorn",  
"gender": "Male",  
"class": "Warrior",  
"faction": "The Iron Brotherhood",  
"startingLocation": "Ironhold",  
"funFact": "Thorn once single-handedly defeated a dragon."
```

- b. Display the list of characters in either a table or in columns and rows of three characters per row.
 - c. Update the navigation in your web application to support bi-direction navigation (navbar links and footer links) to this component.
3. Write unit tests to verify the functionality of the MenuComponent. You should write the tests before you implement the components functionality, following TDD principles. Use the following test cases:
 - a. Should have correct route for PlayersComponent (Player's link leads to the PlayersComponent).
 - b. Should create PlayersComponent.
 - c. Should correctly display a list of characters.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable Player's page. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- Update the app.component.spec.ts files configurations to allow routing.
- Use a TypeScript interface to build an array of character objects.
- Use Angular's @for template syntax to iterate over the array of character objects.
- Give the characters list, in the HTML code, a CSS selector to give you a mechanism for accessing the character items in your unit test.
- Add the "Should have correct route for PlayersComponent" to the app.component.spec.ts file
- Add the other two unit tests to the players.component.spec.ts file.

Chapter 4. Angular Forms I

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Examine the implementation of template-driven forms in Angular.
- Assess the role of form validation in enhancing user experience and data integrity.
- Compare template-driven forms in Angular to standard HTML forms.
- Test for functionality and accuracy in form-related unit tests.

Template-Driven Forms with TDD

Angular provides two approaches for building HTML forms and handling user input in forms: template-driven forms and reactive forms. In this section we will explore how to use template-driven forms to build and collect user data from an HTML form. Template-driven forms use form directives and form models to manipulate web form controls. They are the most straightforward approach to working with web forms and ideal for simple data entry forms. Template-driven forms do not include out of the box strategies for data validation, beyond standard HTML 5 directives and they require a model for data binding.

Continuing with where we left off in the virtual-taco-stand project from the previous chapter:

1. Open the project in your editor.
2. Open the `order.component.spec.ts` file and add unit tests for the following criteria's: should return a random order ID between 1 and 1000 with no decimal places, should add a selected taco to the order with correct customization, should calculate the total price correctly for multiple tacos, should reset all form fields to their default values after `resetForm` is called.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
```

```
import { OrderComponent } from './order.component';
```

```
describe('OrderComponent', () => {  
  let component: OrderComponent;  
  let fixture: ComponentFixture<OrderComponent>;
```

```
  beforeEach(async () => {  
    await TestBed.configureTestingModule({  
      imports: [OrderComponent]  
    })  
    .compileComponents();
```

```

    fixture = TestBed.createComponent(OrderComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should generate a random order ID between 1 and 1000 with no decimal
places', () => {
    component.addToOrder(); // This will trigger the generation of a new order ID
    expect(component.order.orderId).toBeGreaterThan(0);
    expect(component.order.orderId).toBeLessThanOrEqual(1000);
    expect(Number.isInteger(component.order.orderId)).toBe(true);
  });

  it('should add a selected taco to the order with correct customization', () => {
    component.selectedTacoId = 1;
    component.noOnions = true;
    component.noCilantro = false;
    component.quantity = 2;
    component.addToOrder();
    const addedTaco = component.order.tacos[0];

    expect(addedTaco.id).toBe(1);
    expect(addedTaco.quantity).toBe(2);
    expect(addedTaco.noOnions).toBe(true);
    expect(addedTaco.noCilantro).toBe(false);
  });

  it('should calculate the total price correctly for multiple tacos', () => {
    component.order.tacos = [
      { id: 1, name: 'Taco1', price: 5, quantity: 2},
      { id: 2, name: 'Taco2', price: 6.5, quantity: 3}
    ]

    const totalPrice = component.getTotal();
    expect(totalPrice).toBe(29.5);
  });

  it('should reset all form fields to their default values after resetForm is called',
  () => {

```

```

    component.selectedTacoId = 2;
    component.quantity = 3;
    component.noOnions = true;
    component.noCilantro = true;

    component.resetForm();

    expect(component.selectedTacoId).toBe(1);
    expect(component.quantity).toBe(1);
    expect(component.noOnions).toBe(false);
    expect(component.noCilantro).toBe(false);
  })
});

```

In this code example, we define four unit tests for verifying the functionality of the OrderComponent. Let's dive into each part of the code to understand how it works.

- **it('should generate a random order ID between 1 and 1000 with no decimal places', () => { ... }):** This defines a unit test that checks if the auto-generated order ID is a numerical value between 1 and 1000 without decimal places. `component.addToOrder()` is called to trigger the generation of a new order ID.
- **it('should add a selected taco to the order with correct customization', () => { ... }):** This defines a unit test that checks if a selected taco is added to the taco order.
- **it('should add a selected taco to the order with correct customization', () => { ... }):** This defines a unit test that checks if the total price is correctly calculated, given a number of tacos.
- **it('should reset all form fields to their default values after resetForm is called', () => { ... }):** This defines a unit test that checks if all of the form fields in the order form are set to default values when the `resetForm()` method is called.

To ensure our tests fail, run `ng test` from a new CLI window.

Passing Tests (Green):

1. Open the `order.component.ts` file and write the HTML, CSS, and TypeScript code to build an order web form for ordering tacos.

```

export interface Taco {
  id: number;
  name: string;
  price: number;
}

```

```

    noOnions?: boolean;
    noCilantro?: boolean;
    quantity?: number;
  }

  export interface Order {
    tacos: Taco[];
    orderId: number;
  }

  import { Component } from '@angular/core';
  import { FormsModule, NgForm } from '@angular/forms';
  import { CommonModule } from '@angular/common';

  @Component({
    selector: 'app-order',
    standalone: true,
    imports: [FormsModule, CommonModule],
    template: `
      <div class="order-form-container">
        <form class="order-form" #tacoOrderForm="ngForm"
          (ngSubmit)="addToOrder();">
          <h1>Complete the form below to place a new order.</h1>

          <fieldset>
            <legend>My Order</legend>
            <label for="tacoType">Taco Type</label>
            <select name="tacoType" id="tacoType" [(ngModel)]="selectedTacoId"
ngModel>
              @for (taco of tacos; track taco) {
                <option value="{{ taco.id }}">{{ taco.name }}</option>
              }
            </select>

            <label for="qty">Quantity</label>
            <input type="text" id="qty" name="qty" class="qty-input"
[(ngModel)]="quantity" ngModel>

            <div class="customization-section">
              <label>Customize</label>

              <div class="customization-option">
                <input type="checkbox" id="noOnions" name="noOnions"
[(ngModel)]="noOnions" ngModel>

```

```

        <label for="noOnions">No Onions</label>
    </div>

    <div class="customization-option">
        <input type="checkbox" id="noCilantro" name="noCilantro"
        [(ngModel)]="noCilantro" ngModel>
        <label for="noCilantro">No Cilantro</label>
    </div>
</div>

<input type="submit" value="Add to Order" />
</fieldset>
</form>

<div class="order-summary">
    <h1>Order Summary</h1>

    @if (order.tacos.length > 0) {
    <ul>
        @for (taco of order.tacos; track taco) {
        <li>
            <strong>{{ taco.quantity }}x {{ taco.name }}</strong>
            <br />
            Price per taco: {{ taco.price | currency:'USD':'symbol':'1.2-2' }}
            <br />
            @if (taco.noOnions) {
                No onions
            <br />
            }
            @if (taco.noCilantro) {
                No cilantro
            <br />
            }
        </li>
        }
    </ul>
    <p><strong>Total:</strong> {{ getTotal() | currency:'USD':'symbol':'1.2-2'
    }}</p>
    } @else {
    <p>No tacos added to the order yet.</p>
    }
</div>

</div>

```

```

`,
styles: [ `
.order-form-container {
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

.order-form {
  flex: 1;
  margin-right: 20px;
}

.order-summary {
  flex: 1;
}

fieldset {
  margin-bottom: 20px;
}

label, select, qty-input {
  display: block;
  margin-bottom: 5px;
}

.qty-input, select, input[type="submit"] {
  padding: 8px;
  box-sizing: border-box;
}

select {
  width: 100%;
}

.qty-input {
  width: 20%;
}

input[type="submit"] {
  float: right;
}

.customization-section {

```

```

    margin-top: 10px;
  }

  .customization-option {
    display: flex;
    align-items: center;
    margin-bottom: 5px;
  }

  input[type="checkbox"] {
    margin-right: 5px;
  }

  .order-summary li {
    margin-bottom: 10px;
    padding: 5px;
  }
  ,
]
})
export class OrderComponent {
  tacos: Taco[];
  order: Order;
  selectedTacoId: number;
  quantity: number;
  noOnions: boolean = false;
  noCilantro: boolean = false;
  orderTotal: number;

  constructor() {
    this.tacos = [
      {id: 1, name: "Carnitas Taco", price: 3.25},
      {id: 2, name: "Queso Birria Taco", price: 3.50},
      {id: 3, name: "Al Pastor Taco", price: 3.25},
      {id: 4, name: "Tacos de Lengua", price: 3.50},
      {id: 5, name: "Chicken Taco", price: 3.25},
      {id: 6, name: "Fish Taco", price: 3.25},
      {id: 7, name: "Veggie Taco", price: 3.25},
      {id: 8, name: "Chicharron Taco", price: 3.25},
      {id: 9, name: "Potato Taco", price: 3.25},
      {id: 10, name: "Chorizo Taco", price: 3.25}
    ];

    this.order = { tacos: [], orderId: 0 };
  }
}

```

```

    this.selectedTacold = this.tacos[0].id;
    this.quantity = 1;
    this.orderTotal = 0;
  }

  addToOrder() {
    const selectedTacoNum = Number(this.selectedTacold);

    const selectedTaco = this.tacos.find(taco => taco.id === selectedTacoNum);

    // random number between 1 and 1000 for order Id no decimal places
    this.order.orderId = Math.floor(Math.random() * 1000) + 1;

    if (selectedTaco !== undefined) {
      const tacoToAdd = {
        id: selectedTaco.id,
        name: selectedTaco.name,
        price: selectedTaco.price,
        noOnions: this.noOnions,
        noCilantro: this.noCilantro,
        quantity: this.quantity
      }

      this.order.tacos.push(tacoToAdd);
      console.log('Order after adding:', this.order);

      this.resetForm();
    } else {
      console.error('Taco not found in the list of available tacos.',
this.selectedTacold)
    }
  }

  getTotal() {
    return this.order.tacos.reduce((acc, taco) => acc + (taco.price * (taco.quantity
?? 1)), 0);
  }

  resetForm () {
    if (this.tacos.length > 0) {
      this.selectedTacold = this.tacos[0].id;
    }

    this.quantity = 1;

```

```

    this.noOnions = false;
    this.noCilantro = false;
  }
}

```

In this code example, we are using template-driven forms and TypeScript interfaces to build an order form for our virtual-taco-stand application. Let's take a closer look at each part of the code to understand how it works.

- **export interface Taco {}:** This block of code creates a TypeScript interface for a taco, which will be used in the order form. Of note, we are using TypeScript's "Optional Property" for the noOnions, noCilantro, and quantity properties. The "Optional Property" operator tells TypeScript these properties are optional in the Taco interface. Meaning, an object of type Taco can be created without providing values for each of these properties.
- **export interface Order {}:** This block of code creates a TypeScript interface for storing the tacos placed in an order.
- **<form class="order-form" #tacoOrderForm="ngForm" (ngSubmit)="addToOrder();">:** This line of code creates a web form, declares it as a template-driven form, and assigns the method addToOrder() to the forms submission (called when the form is submitted).
- **<select name="tacoType" id="tacoType" [(ngModel)]="selectedTacold" ngModel>:** This line of code creates a select menu control and instructs Angular to use the property selectedTacold from the OrderComponent class. The syntax [(ngModel)]="" is needed to map the selectable item to the selectedTacold property. Most developers refer to the syntax "[()]" as "banana in a box."
- **<input type="checkbox" id="noOnions" name="noOnions" [(ngModel)]="noOnions" ngModel>:** This line of code creates an input control and instructs Angular to use the property noOnions from the OrderComponent class as the binding model property. Using the directive ngModel tells Angular to treat this field as a model property value using template-driven forms.
- **Price per taco: {{ taco.price | currency:'USD':'symbol':'1.2-2' }}:** The symbol | is called a pipe operator, allowing us to transform strings into predefined data formats. Built-in pipes include: DatePipe, UpperCasePipe, LowerCasePipe, CurrencyPipe, DecimalPipe, PercentPipe, AsyncPipe, and JsonPipe,
- **return this.order.tacos.reduce((acc, taco) => acc + (taco.price * (taco.quantity ?? 1)), 0);** This line of code calculates the total price of all tacos in an order. It

iterates over the tacos array in the order object using JavaScript's built-in reduce method. Each taco in the array is multiplied by the taco's price and quantity. If the quantity is null or undefined, it is defaulted to 1 using the nullish coalescing operator. The nullish coalescing operator (??) is a logical operator in JavaScript that returns its right-sided operand when the left-hand side operand is null or undefined. Using this operator is a great way to provide default values for variables and expressions.

- **this.resetForm():** This line of code resets the order forms values. The functionality is similar to how the reset input type in an HTML input form control behaves.
- **this.order.tacos.push(tacoToAdd):** This line of code adds the tacoToAdd object literal to the tacos array in the order object.

Rerun the unit tests to ensure they are passing (ng test) and start the server to test the OrderComponent. You should see an order form and have the ability to order tacos. Each ordered taco should appear in a section next to the form as a running total of selected tacos with a final total.

Unit Test Results:

```
✓ Browser application bundle generation complete.
17 07 2024 14:56:10.441:WARN [karma]: No captured browser
17 07 2024 14:56:10.451:INFO [karma-server]: Karma v6.4.
17 07 2024 14:56:10.451:INFO [launcher]: Launching brows
17 07 2024 14:56:10.453:INFO [launcher]: Starting browse
17 07 2024 14:56:11.419:INFO [Firefox 128.0 (Mac OS 10.1
LOG: 'Order after adding:', Object{tacos: [Object{id: ..
Firefox 128.0 (Mac OS 10.15): Executed 9 of 13 SUCCESS (
LOG: 'Order after adding:', Object{tacos: [Object{id: ..
Firefox 128.0 (Mac OS 10.15): Executed 10 of 13 SUCCESS
Firefox 128.0 (Mac OS 10.15): Executed 13 of 13 SUCCESS
TOTAL: 13 SUCCESS
█
```

OrderComponent:

Complete the form below to place a new order.

My Order

Taco Type

Carnitas Taco

Quantity

1

Customize

☐ No Onions

☐ No Cilantro

Add to Order

Order Summary

- **2x Al Pastor Taco**
Price per taco: \$3.25
No onions
- **1x Fish Taco**
Price per taco: \$3.25
No cilantro
- Total: \$9.75**

Hands-On 4.1: TDD and Template-Driven Forms

In this hands-on project, you will be extending the rpg-character-builder by building the CreateCharacter component. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter's hands-on project.
2. Write the HTML, CSS, and TypeScript code for the CreateCharacter component. At minimum, you must include the following:
 - a. A template-driven form to create new characters. The form must include the following form controls: `name`, `gender` (make this a select menu) with options for "Male, Female, and Other," and `class` (make this a select menu) with options for "Warrior, Mage, and Rogue." In other words, the form should have one input of type text, two select menus, and a submit button.
 - b. Display the created characters in a separate section within the HTML. Use a table, unordered list, or cards with columns and rows to display the data.
 - c. Update the navigation in your web application to support bi-direction navigation (navbar links and footer links) to this component.
3. Write unit tests to verify the functionality of the CreateCharacter component. You should write the tests before you implement the components functionality, following TDD principles. Use the following test cases:

- a. Test 1: should generate a random character ID between 1 and 1000 with no decimal places.
- b. Test 2: should add a character with correct customization.
- c. Test 3: should reset all form fields to their default values after resetForm is called.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable Player's page. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- Use template-driven forms to build the create character form.
- Use a TypeScript interface for the form model.
- Use a method to handle the form submission.
- Reference the virtual-taco-stand projects OrderComponent customization-section (HTML div) for assistance with building the create character form.
- Use an array to store the created characters. For example, **characters: Character[];**
- Use Angular's @for template syntax to iterate over the characters array.
- Call this.resetForm() after you've added the newly submitted character to the characters array.

Chapter 5. Angular Forms II

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Explore the implementation of reactive forms in Angular.
- Evaluate the use of guards for route protection and their impact on application security.
- Assess the role of validation in reactive forms for user experience and data integrity.
- Analyze the integration of authentication services with reactive forms.

Reactive Forms with TDD

In this section we will take a look at how we can leverage reactive forms in Angular to build a SigninComponent, following TDD principles.

Continuing where we left off from in the previous chapter's virtual-taco-stand project:

1. Open the virtual-taco-stand project in your editor.
2. Open a new terminal window and using Angular's CLI tool, generate the following files:
 - a. `ng generate service auth`
 - b. `ng generate guard auth`
3. Open the `auth.service.spec.ts` file and add a unit tests that check if `authState` is true for successful sign in and false for unsuccessful sign in.

```
import { TestBed } from '@angular/core/testing';
import { AuthService } from './auth.service';
import { CookieService } from 'ngx-cookie-service';

describe('AuthService', () => {
  let service: AuthService;
  let cookieServiceSpy: jasmine.SpyObj<CookieService>;

  beforeEach(() => {
    const spy = jasmine.createSpyObj('CookieService', ['set', 'deleteAll']);
    TestBed.configureTestingModule({
      providers: [AuthService, { provide: CookieService, useValue: spy }]
    });
    service = TestBed.inject(AuthService);
    cookieServiceSpy = TestBed.inject(CookieService) as
    jasmine.SpyObj<CookieService>;
  });

  it('should be created', () => {
    expect(service).toBeTruthy();
  });

  it('should set cookie and authState to true on successful signin', () => {
    const result = service.signin('wizardlywand@hogwarts.com', 'Alohomora123');
    expect(result).toBeTrue();
    expect(service.getAuthState().subscribe(state => expect(state).toBeTrue()));
    expect(cookieServiceSpy.set.calls.count()).toBe(1);
  });

  it('should not set cookie and authState to true on unsuccessful signin', () => {
```

```

    const result = service.signin('wrongemail@hogwarts.com', 'wrongpassword');
    expect(result).toBeFalse();
    expect(service.getAuthState().subscribe(state => expect(state).toBeFalse()));
    expect(cookieServiceSpy.set.calls.count()).toBe(0);
  });
});

```

In this code example, we create two unit tests: should set cookie and authState to true on successful signin and should not set cookie and authState to true on unsuccessful signin. Let's take a look at each part of the code to see how it works.

- **it('should set cookie and authState to true on successful signin', () => { ... }):** In this unit test, we call the `signin()` method from the `AuthService` to test the functionality of our signin service with valid credentials. `Result` represents the response from the signin method and `toBeTrue()` is called to test the 'truthy' of the result variable. Next, the `getAuthState()` method is subscribed to and tested `.toBeTrue()`.
- **it('should not set cookie and authState to true on unsuccessful signin', () => { ... }):** In this unit test, we call the `signin()` method with invalid credentials. The code is similar to what we used in the previous unit test, but in reverse (falsey conditions).

To ensure our tests fail, run `ng test` from a new CLI window.

Passing Test (Green):

1. Open the `auth.service.ts` file and add the following code:

```

export interface User {
  empId: number;
  email: string;
  password: string;
}

import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';
import { CookieService } from 'ngx-cookie-service';
import { Router } from '@angular/router';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private users: User[];

```

```

private authState = new BehaviorSubject(<boolean>false);

constructor(private cookieService: CookieService, private router: Router) {
  this.users = [
    { empld: 1007, email: 'wizardlywand@hogwarts.com', password:
'Alohomora123' },
    { empld: 1008, email: 'quidditchqueen@hogwarts.com', password:
'Quaffle22' },
    { empld: 1009, email: 'potionmaster@hogwarts.com', password: 'Polyjuice99'
},
    { empld: 1010, email: 'mugglemania@hogwarts.com', password: 'Dementor0'
},
    { empld: 1011, email: 'spellbinder@hogwarts.com', password:
'Expelliarmus88' }
  ];
}

getAuthState() {
  return this.authState.asObservable();
}

signin(email: string, password: string) {
  const user = this.users.find(user => user.email === email && user.password
=== password);

  if (user) {
    this.cookieService.set('session_user', email, 1);
    this.authState.next(true);
    return true;
  } else {
    this.authState.next(false);
    return false;
  }
}

signout() {
  this.cookieService.deleteAll();
  this.authState.next(false);
  this.router.navigate(['/signin']).then(() => {});
}
}

```

In this code example, we create a service with several methods for handling user authentication in our virtual-taco-stand application. Let's take a closer look at each part of the service to understand how it works.

- **export interface User { ... }:** This block of code defines a TypeScript interface named User with properties for empId, email, and password. We will use this interface to build an array of user objects for authenticating authorized users.
- **private authState = new BehaviorSubject(<Boolean>false):** This line of code defines a variable named authState as a BehaviorSubject object. BehaviorSubject is a type of subject from the RxJS library, which is used to emit values to subscribing observers. In other words, when you subscribe to it, it will immediately emit a value. We are using this to keep track of whether someone is signed into our application or not.
- **constructor(private cookieService: CookieService, private router: Router) { ... }:** This block of code creates a constructor with actual parameters for the CookieService and Router module. Configuring our constructor in this way allows us access to the module's functionality within this component. The technical term is "Dependency Injection," but in simpler terms, it grants us access to the module.
- **getAuthState() { ... }:** This is a method that returns the authState BehaviorSubject as an [observable](#).
- **signin(email: string, password: string) {...}:** This is a method that we use to sign a user into our Angular application. The first expression in the method uses JavaScript's built-in find() function to compare the formal parameters against the users array. If a match is found it means the user is "authenticated" and should be granted access to the application. To do this, we use the npm package "ngx-cookie-service" to create and assign a cookie to the user's browser. If authentication fails (email and password do not match an object in the array), we set the authState to false and return false.
- **signout():** This method is used to mimic the behavior of signing a user out of the application. To do this, we call the deleteAll() method from the "ngx-cookie-service," set the authState to false, and redirect users to the /signin route.

2. Open the auth.guard.ts file and add the following code:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { CookieService } from 'ngx-cookie-service';
```

```

export const authGuard: CanActivateFn = (route, state) => {
  const cookieService = inject(CookieService);

  if (cookieService.get('session_user')) {
    return true;
  } else {
    const router = inject(Router);
    router.navigate(['/signin'], {queryParams: {returnUrl: state.url}});
    return false;
  }
};

```

In this code example, we build an Angular guard to check if a user is authenticated (signed in). Let's take a look at each part of the code to understand how it works.

- **const cookieService = inject(CookieService):** This line of code is using Angular's inject function to give us access to the "ngx-cookie-service." This approach is similar to the previous example where we used formal parameters in the component's constructor. The only difference is we are not using a constructor for the dependency injection.
- **if(cookieService.get('session_user')) { ... }:** This block of code is checking the user's browser for a cookie named "session_user." If it is present, it means the user has been authenticated (signed in) and should be given access to the requested route. For non-authenticated users, they are redirected back to the /signin route.

Rerun the unit tests to ensure they are passing (ng test).

Failing Test (Red):

1. Open the signin.component.spec.ts file and write a unit test to confirm the signin method is called on form submission.

```

import { ComponentFixture, TestBed } from '@angular/core/testing';
import { RouterTestingModule } from '@angular/router/testing';
import { SigninComponent } from './signin.component';
import { ReactiveFormsModule } from '@angular/forms';
import { AuthService } from '../auth.service';
import { of } from 'rxjs';

class MockAuthService {
  signin(email: string, password: string) {
    return of(true);
  }
}

```

```

describe('SigninComponent', () => {
  let component: SigninComponent;
  let fixture: ComponentFixture<SigninComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [RouterTestingModule, SigninComponent, ReactiveFormsModule],
      providers: [{ provide: AuthService, useClass: MockAuthService }]
    })
    .compileComponents();

    fixture = TestBed.createComponent(SigninComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should call signin method on form submit', () => {
    const signInSpy = spyOn((component as any).authService,
'signin').and.callThrough();
    component.signinForm.controls['email'].setValue('test@example.com');
    component.signinForm.controls['password'].setValue('Password123');
    component.signin();
    expect(signInSpy).toHaveBeenCalledWith('test@example.com',
'Password123');
  });
});

```

In this code example, we define a single unit test to check if the signin method is called on form submissions. Let's take a closer look at the functionality of this unit test.

- **spyOn((component as any).authService, 'signin').and.callThrough()):** This line of code creates a spy and uses it to “spy on” the SgininComponent’s authService. This is needed to setup the sign in form and to assign values to the sign in form controls.
- **component.signin():** This line of code calls the signin() method from the SigninComponent.

- **expect(signInSpy).toHaveBeenCalledWith('test@example.com', 'Password123');** This line of code checks if the `signin()` method was called with the values we assigned to the email and password input fields.

To ensure the unit tests fail, run `ng test` from a new CLI window.

Passing Test (Green):

1. Open `signin.component.ts` and write the HTML, CSS, and TypeScript for the sign in page.

```
import { Component } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { CommonModule } from '@angular/common';
import { AuthService } from '../auth.service';

@Component({
  selector: 'app-signin',
  standalone: true,
  imports: [ReactiveFormsModule, CommonModule],
  template: `
    <div class="signin-form-container">
      <form [formGroup]="signInForm" (ngSubmit)="signin();" class="signin-form">
        <h1>Complete the form below to sign in.</h1>

        <fieldset>
          <legend>User Sign In</legend>

          <label for="email">Email</label>
          <input formControlName="email" type="email" id="email" name="email">
          @if (signInForm.controls['email'].touched &&
signInForm.controls['email'].hasError('required')) {
            <small class="error">Email is required.</small>
          }
          @if (signInForm.controls['email'].touched &&
signInForm.controls['email'].hasError('email')) {
            <small class="error">Invalid email address.</small>
          }

          <label for="password">Password</label>
          <input formControlName="password" id="password" type="password">
```

```

    @if (signinForm.controls['password'].touched &&
signinForm.controls['password'].hasError('required')) {
      <small class="error">Password is required.</small>
    }
    @if (signinForm.controls['password'].touched &&
signinForm.controls['password'].hasError('pattern')) {
      <small class="error">Password must be at least 8 characters long and
contain at least one uppercase letter and one number.</small>
    }
    <input type="submit" [disabled]="!signinForm.valid" value="Sign In">
  </fieldset>
</form>
</div>
`,
styles: `
.signin-form-container {
  display: flex;
  justify-content: space-between;
  width: 50%;
}

.signin-form {
  flex: 1;
  margin-right: 20px;
}

label, input {
  display: block;
  margin-bottom: 5px;
}

input, input[type="submit"] {
  padding: 8px;
  box-sizing: border-box;
}

input[type="email"], input[type="password"] {
  width: 100%;
}

input[type="submit"] {
  margin-top: 10px;
  float: right;
}

```

```

        .error {
            color: red;
            padding: 5px;
        }
    },
    })
    export class SigninComponent {
        signinForm: FormGroup = this.fb.group({
            email: [null, Validators.compose([Validators.required, Validators.email])],
            password: [null, Validators.compose([Validators.required,
Validators.pattern(/^(?=.*[A-Z])(?=.*[0-9]).{8,}$/)])]
        });

        constructor(
            private fb: FormBuilder,
            private router: Router,
            private route: ActivatedRoute,
            private authService: AuthService) {
        }

        signin() {
            const email = this.signinForm.controls['email'].value;
            const password = this.signinForm.controls['password'].value;

            if (this.authService.signin(email, password)) {
                const returnUrl = this.route.snapshot.queryParamMap.get('returnUrl') || '/';
                this.router.navigate([returnUrl]);
            } else {
                alert('Invalid email or password. Please try again.');
```

In this code example, we use reactive forms to build a sign in form and define a `signin()` method to handle form submissions. Let's take a closer look at how this code works.

- **signinForm: FormGroup = this.fb.group({ ... }):** This block of code uses Angular's reactive form to build a form with two controls: email and password. Angular includes data validation through the `Validators` module. To use validators, you add the validation checks to the `Validators.compose` array. The most commonly used validators are: `required`, `email`, `minLength`, `maxLength`, and `pattern`.

- **signin():** This block of code defines a method for processing form submissions. To retrieve the values from a reactive form you call `this.formName.controls['controlName'].value`. Where `this.formName` represents the name of the form you defined and `.controls['controlName']` represents the name of the control you defined. For example, `this.signinForm.controls['email'].value`. The property `.value` is where the inputted value is stored. You can also manually set the from controls value by calling `.setValue('value-to-set');`
 - **this.route.snapshot.queryParamMap.get('returnUrl') || '/':** This line of code is checking the URL for a parameter value named “returnUrl.” If the route parameter is present, it is used to redirect users to the value. Otherwise, a default value of “root,” which is represented by a “/” is used for the redirection.
2. Open the `app.component.ts` file and update the code to show the users name for authenticated users with a sign out option and a sign in option for non-authenticated users.

```
import { Component } from '@angular/core';
import { RouterLink, RouterOutlet } from '@angular/router';
import { CookieService } from 'ngx-cookie-service';
import { AuthService } from './auth.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, RouterLink],
  template: `
    <div class="wrapper">
      <header class="banner">
        
      </header>

      <div class="sign-in-container">
        @if (email) {
          <p>Welcome, {{ email }}!</p>
          <button (click)="signout()">Sign Out</button>
        } @else {
          <a routerLink="/signin" class="sign-in-link">Sign In</a>
        }
      </div>

      <main class="main-content">
        <nav class="navbar">
```

```

    <ul>
      <li><a routerLink="/">Home</a></li>
      <li><a routerLink="/menu">Menu</a></li>
      <li><a routerLink="/order">Order</a></li>
      <li><a routerLink="/daily-specials">Daily Specials</a></li>
      <li><a routerLink="/feedback">Feedback</a></li>
    </ul>
  </nav>
  <section class="content">
    <router-outlet />
  </section>
</main>

<footer class="footer">
  <nav class="footer-nav">
    <a routerLink="/">Home</a> |
    <a routerLink="/menu">Menu</a> |
    <a routerLink="/order">Order</a> |
    <a routerLink="/daily-specials">Daily Specials</a> |
    <a routerLink="/feedback">Feedback</a>
  </nav>
  <p>&copy; 2024 Virtual Taco Stand</p>
</footer>
</div>
`,
styles: [
  `
    .sign-in-container {
      text-align: right;
      padding-right: 20px;
      margin-top: 10px;
    }

    .sign-in-link {
      color: #000000;
      text-decoration: none;
      font-family: 'Lato', sans-serif;
    }

    .sign-in-link:hover {
      text-decoration: underline;
    }
  `,
],
]

```

```

  })
  export class AppComponent {
    email?: string;

    constructor(private authService: AuthService, private cookieService:
CookieService) {
    }

    ngOnInit() {
      this.authService.getAuthState().subscribe((isAuth) => {
        if (isAuth) {
          this.email = this.cookieService.get('session_user');
        }
      });
    }

    signout() {
      this.authService.signout();
    }
  }

```

In this code example, the AppComponent is updated to account for authenticated and non-authenticated users. For authenticated users, we display a welcome message with their email address and an option to sign out. For non-authenticated users, we display a sign in link. Let's take a closer look at each part of the code to understand how it works.

- **this.authService.getAuthState().subscribe((isAuth) => { ... })**: This block of code subscribes to the getAuthState() method in the AuthService to determine if the user is signed in (authenticated) or not (non-authenticated). The ngx-cookie-service is used to capture the authenticated user's email address to use in the HTML code.
 - **signout()**: This method is used to call the signout() method from the AuthService to sign users out of the application.
3. Open the app.routes.ts file and add the authGuard to the canActivate property of the order route.

```

import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { MenuComponent } from './menu/menu.component';
import { OrderComponent } from './order/order.component';
import { DailySpecialsComponent } from './daily-specials/daily-
specials.component';
import { FeedbackComponent } from './feedback/feedback.component';

```

```

import { SigninComponent } from './signin/signin.component';
import { authGuard } from './auth.guard';

export const routes: Routes = [
  {
    path: '',
    component: HomeComponent
  },
  {
    path: 'home',
    component: HomeComponent
  },
  {
    path: 'menu',
    component: MenuComponent
  },
  {
    path: 'order',
    component: OrderComponent,
    canActivate: [authGuard]
  },
  {
    path: 'daily-specials',
    component: DailySpecialsComponent
  },
  {
    path: 'feedback',
    component: FeedbackComponent
  },
  {
    path: 'signin',
    component: SigninComponent
  }
];

```

In this code example, we add the AuthGuard to the OrderComponent, ensuring only authenticated users can place orders.

Rerun the unit tests to ensure they are passing (ng test):

```
✓ Browser application bundle generation complete.
18 07 2024 14:47:38.776:WARN [karma]: No captured browser, open ht
18 07 2024 14:47:38.784:INFO [karma-server]: Karma v6.4.3 server s
18 07 2024 14:47:38.784:INFO [launcher]: Launching browsers Firefo
18 07 2024 14:47:38.786:INFO [launcher]: Starting browser Firefox
18 07 2024 14:47:39.936:INFO [Firefox 128.0 (Mac OS 10.15)]: Conne
LOG: 'Order after adding:', Object{tacos: [Object{id: ..., name: .
Firefox 128.0 (Mac OS 10.15): Executed 10 of 17 SUCCESS (0 secs /
Firefox 128.0 (Mac OS 10.15): Executed 17 of 17 SUCCESS (0.115 sec
TOTAL: 17 SUCCESS
```

Start the project (ng serve) to verify the functionality of the sign in component and the order component. You should not be able to access the OrderComponent unless you are signed in.

Clicking on the “Order” link in the navigation menu:

Complete the form below to sign in.

User Sign In

Email

Password

Sign In

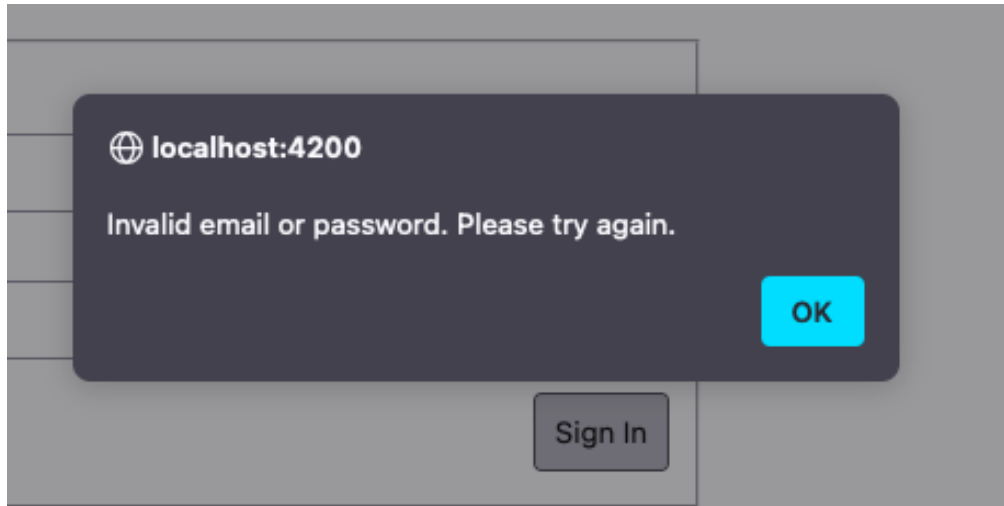
Landing page after successfully signing in (located under the banner and to the right).



Welcome, wizardlywand@hogwarts.com!

Sign Out

Error message for invalid email and/or password:



Hands-On 5.1: TDD and Reactive Forms

In this hands-on project, you will be extending the rpg-character-builder by building the Signin component. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter's hands-on project.
2. Write the HTML, CSS, and TypeScript code for the Signin component. At minimum, you must include the following:
 - a. An Angular reactive form to the signin form with form controls for email and password. Make these controls required and use validators for email and password (use the validators pattern from the virtual-taco-stand project).
 - b. Use the AuthService signin() method to sign user in. This method returns true or false, so you can use it in an if...else statement. For example,
if (this.authService.signin(email, password)) { ... } else { ... }
 - c. Display an alert box with a generic message for non-authenticated users.
 - d. Update the navigation in your web application to support bi-direction navigation (navbar links and footer links) to this component.

3. Build an Angular service to manage the state of the authentication (used in the AppComponent for signing in and out of the application). Name the service “auth.” In this service include the following:
 - a. **private users: User[]**: Array of user objects with properties for empld, email, and password.
 - b. **private authState**: declared as a new BehaviorSubject.
 - c. **getAuthState()**: method that returns this.authState as an observable.
 - d. **signin(ema: string, password: string)**: Method for signing users into the application. If authenticated (located in the user’s array), use the ‘ngx-cookie-service’ package to add a cookie named ‘session_user’ to the user’s browser. Set the authState.next() to either true or false, depending on whether they were authenticated (located in the user’s array). This method should return true or false; true for authenticated users and false for non-authenticated users.
 - e. **signout()**: method for deleting all cookies added to the users browser and setting the autState to false. Redirect signed out users to the /signin route.
4. Create an AuthGuard to prevent users from accessing the CreateCharacter component, unless they are signed in. The AuthGuard should return true if the ‘session_user’ cookie is present and false with a redirect back to the SignInComponent if not.
5. Add the AuthGuard to the CreateCharacterComponent path in the app.routes.ts, under the canActivate property. For example,


```
{
  path: create-character,
  component: CreateCharacterComponent,
  canActivate: [authGuard]
},
```
6. Update the AppComponent with logic to display “Welcome [email]” with a “Sign out” button for signed in users and a “Sign In” button for non-signed in users. In the AppComponent class use the ngOnInit() method to call the authService.getAuthState() observable to determine if a user is authenticated. Additionally, add a signout() method that calls the authSevice.signout() method.

7. Write unit tests to verify the functionality of the Signin component. You should write the tests before you implement the components functionality, following TDD principles. Use the following test cases:
 - a. Test 1: Should set cookie and authState to true on successful sign in.
 - b. Test 2: Should not set cookie and authState to true on unsuccessful sign in
 - c. Test 3: Should call signin method on form submission.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable Player's page. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- Use reactive forms to build the sign in form.
- Use an interface in the auth.service.ts file to build the users array. Include properties for empId, email, and password.
- Use the 'ngx-cookie-service' to create the 'session_user' cookie.
- Use JavaScript's built-in find() function to authenticate users (exists in the users array).
- Use an AuthService class to manage the authentication state.
- Use a variable in the AppComponent class to determine if a user is authenticated. For example,

```
ngOnInit() {  
  this.authService.getAuthState().subscribe((isAuth) => {  
    if (isAuth) {  
      this.email = this.cookieService.get('session_user');  
    }  
  });  
}
```

- Use the virtual-taco-stand project as a reference for building the AuthGuard, AuthService, and SigninComponent.

Chapter 6. Angular Forms III

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Discover the capabilities of dynamic forms in Angular.
- Evaluate the use of advanced form controls in enhancing user interactions.
- Test for the validity and functionality of dynamic forms through unit testing.
- Explain the impact of dynamic forms on application functionality compared to traditional forms.

Dynamic Forms and TDD

In this section, we will take a look at how to build dynamic web forms in Angular. Dynamic web forms are a strategy for building forms that are driven by data. That means, we use data to drive the structure, layout, and functionality of an Angular web form. Dynamic forms in Angular, are similar to reactive forms, but with the caveat that the form is rendered “on the fly.”

The advantages of dynamic forms include:

1. **Flexibility:** Dynamic forms are more flexible because the form is being driven from data stored in either a database, file, or some other type of data storage mechanism. This allows us to generate forms quickly and on the fly by simply adding new data to our data storage device. Imagine for a moment we are building a quality assurance application for a customer service company. And, this company regularly requests new types of forms to be added to their application. We have two approaches for satisfying their requirements. Option 1 is to build a new Angular component each time a new form is requested. Option 2 is to build a single dynamic form and save the structure of the form in a database. With option 2, each time a new form is requested, we simply add a new entry in our database with the request form’s structure. Whereas, with option 1, we would have to write the Angular code to build a new form, write unit tests to test the new form, and then deploy the application to production.
2. **Reusability:** Dynamic forms promote reusability because we can create generic components and services to generate form controls and validation dynamically, reducing the amount of boiler plate code we have throughout our web application.
3. **Scalability:** Dynamic forms are easier to manage because the structure of the form is stored in a database. This makes scalability easier and more efficient.

Continuing with where we left off in the previous chapter’s virtual-taco-stand project:

1. Open the `feedback.component.spec.ts` file and write unit tests to check if the form is invalid when empty, if the form is valid when filled correctly, and if the `leaveFeedback()` method is called on form submission with valid data.

```

import { ComponentFixture, TestBed } from '@angular/core/testing';

import { FeedbackComponent } from './feedback.component';
import { FormsModule, ReactiveFormsModule } from '@angular/forms';
import { By } from '@angular/platform-browser';

describe('FeedbackComponent', () => {
  let component: FeedbackComponent;
  let fixture: ComponentFixture<FeedbackComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [FeedbackComponent, ReactiveFormsModule, FormsModule]
    })
    .compileComponents();

    fixture = TestBed.createComponent(FeedbackComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('form should be invalid when empty', () => {
    expect(component.feedbackForm.valid).toBeFalsy();
  });

  it('form should be valid when filled correctly', () => {
    component.feedbackForm.controls['rating'].setValue('5');
    component.feedbackForm.controls['recommend'].setValue('Yes');
    component.feedbackForm.controls['comments'].setValue('Great service!');
    expect(component.feedbackForm.valid).toBeTruthy();
  });

  it('should call leaveFeedback on form submit with valid data', () => {
    spyOn(component, 'leaveFeedback');

    component.feedbackForm.controls['rating'].setValue('5');
    component.feedbackForm.controls['recommend'].setValue('Yes');
    component.feedbackForm.controls['comments'].setValue('Great service!');
    fixture.detectChanges();
  });

```

```

const form = fixture.debugElement.query(By.css('form'));
form.triggerEventHandler('ngSubmit', null);

expect(component.leaveFeedback).toHaveBeenCalled();
});
});

```

In this code example, we define three unit tests: form should be invalid when empty, form should be valid when filled correctly, and should call leaveFeedback on form submit with valid data. Let's take a closer look at each part of the code to see how it works.

- **it('form should be invalid when empty', () => { ... }):** This unit test calls the .valid() method to check if the form is valid when empty, which is using reactive form validators to check the validity of the feedbackForm.
- **it('form should be valid when filled correctly', () => { ... }):** This unit test assigns values to the feedbackForm controls and calls the .valid() method to check the validity of the feedbackForm.
- **it('should call leaveFeedback on form submit with valid data', () => { ... }):** This unit test checks if the leaveFeedback() method is called when the feedbackForm is submitted.

To ensure our tests are failing, run ng serve from a new CLI window.

Passing Test (Green):

1. Write the HTML, CSS, and TypeScript code for the FeedbackComponent to pass the defined unit tests.

```

import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators, FormArray,
ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-feedback',
  standalone: true,
  imports: [ReactiveFormsModule, CommonModule],
  template: `
    <div class="feedback-form-container">
      <form [formGroup]="feedbackForm" class="feedback-form"
(ngSubmit)="leaveFeedback(); feedbackForm.reset();">

```

```

<h1>Complete the form below to leave feedback.</h1>
<fieldset>
  <legend>Feedback Form</legend>

  <label>Rate our service</label>
  @for(rating of ratings; track rating) {
    <input type="radio" [value]="rating" FormControlName="rating"> {{ rating
  }} <br />
  }

  <label>What did you like about our service?</label>
  <div formArrayName="likes">
    @for(like of likesArray.controls; track like; let i = $index) {
      <input type="checkbox" [FormControlName]="i"> {{ likes[i] }} <br />
    }
  </div>

  <label>What you recommend us?</label>
  <select FormControlName="recommend">
    @for(option of recommendedOptions; track option) {
      <option [value]="option">{{ option }}</option>
    }
  </select>

  <label>Any additional comments?</label>
  <textarea rows="10" FormControlName="comments"></textarea>

  <input type="submit" [disabled]="!feedbackForm.valid" value="Leave
Feedback">
</fieldset>
</form>

<div class="feedback">
  <h1>Customer Feedback</h1>
  <div class=feedback-container>
    @for(feedback of preexistingFeedback; track feedback) {
      <div class="feedback-card">
        <h2>Rating: {{ feedback.rating }}</h2>
        <h3>Likes:</h3>
        <ul class="likes-list">
          @for(like of feedback.likes; track like) {
            <li>{{ like }}</li>
          }
        </ul>
      </div>
    }
  </div>

```

```

        <h3>Would you recommend us?</h3>
        <p>{{ feedback.recommend }}</p>
        <h3>Comments:</h3>
        <p>{{ feedback.comments }}</p>
    </div>
}
</div>
</div>
</div>
`
,

```

styles: `

```

.feedback-form-container {
    display: flex;
    flex-direction: column;
    width: 80%;
    align-items: center;
}

```

```

.feedback-form, .feedback {
    width: 100%;
    margin-bottom: 20px;
}

```

```

.feedback-container {
    display: flex;
    flex-direction: row;
    flex-wrap: wrap;
    justify-content: space-between;
    gap: 20px;
}

```

```

.feedback-card {
    flex: 0 0 calc(50% - 20px);
    box-sizing: border-box;
    border: 1px solid #ccc;
    border-radius: 5px;
    padding: 20px;
    margin: 10px 0;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

```

```

.likes-list {
    list-style-type: none;
    padding: 0;
}

```

```

}

.likes-list li {
  padding: 5px 0;
}

label {
  display: block;
  margin-bottom: 5px;
}

label:first-of-type {
  margin-top: 0;
}

label:not(:first-of-type) {
  margin-top: 10px;
}

select {
  width: 20%;
  display: block;
  margin-bottom: 5px;
  padding: 8px;
  box-sizing: border-box;
}

textarea {
  width: 100%;
  margin-bottom: 5px;
  padding: 8px;
  box-sizing: border-box;
}

input[type="submit"] {
  display: block;
  padding: 8px;
  margin-bottom: 10px;
  box-sizing: border-box;
  float: right;
}

input[type="checkbox"], input[type="radio"] {
  box-sizing: border-box;
}

```

```

    margin-bottom: 10px;
  }

  fieldset {
    margin-bottom: 20px;
  }
,
}))
export class FeedbackComponent {
  ratings: string[] = ['1', '2', '3', '4', '5'];
  likes: string[] = ['Service', 'Quality', 'Price', 'Ambience', 'Other'];
  recommendedOptions: string[] = ['Yes', 'No'];
  preexistingFeedback: any;

  feedbackForm: FormGroup = this.fb.group({
    rating: [null, Validators.compose([Validators.required])],
    likes: this.fb.array(this.likes.map(() => false)),
    recommend: [null, Validators.compose([Validators.required])],
    comments: [null]
  })

  constructor(private fb: FormBuilder) {
    this.preexistingFeedback = [
      {
        rating: '5',
        likes: ['Service', 'Quality'],
        recommend: 'Yes',
        comments: 'Everything was perfect, from the service to the quality of the
products.'
      },
      {
        rating: '4',
        likes: ['Price'],
        recommend: 'Yes',
        comments: 'Great prices and good service, will definitely recommend to
friends.'
      },
      {
        rating: '3',
        likes: ['Ambience'],
        recommend: 'No',
        comments: 'The ambience was nice, but the service could be improved.'
      },
      {

```

```

        rating: '2',
        likes: ['Service'],
        recommend: 'No',
        comments: 'The service was slow, and the quality of the food was not up to
the mark.'
    },
    {
        rating: '5',
        likes: ['Service', 'Quality', 'Price', 'Ambience'],
        recommend: 'Yes',
        comments: 'Absolutely loved everything! Great value for money and friendly
staff.'
    }
];
}

```

```

get likesArray() {
    return this.feedbackForm.get('likes') as FormArray;
}

```

```

leaveFeedback() {
    // Get the boolean values for each checkbox from the FormArray
    const selectedLikesValues = this.likesArray.value;

    // Map and filter the likes based on the boolean values
    const selectedLikes = this.likes
        .map((like, index) => selectedLikesValues[index] ? like : null)
        .filter(like => like !== null);

    const newFeedback = {
        rating: this.feedbackForm.value.rating,
        likes: selectedLikes,
        recommend: this.feedbackForm.value.recommend,
        comments: this.feedbackForm.value.comments
    };

    // Now, selectedLikes contains the actual items that were selected
    console.log('Selected likes:', selectedLikes);
    console.log('Complete form value:', newFeedback);

    this.preexistingFeedback.push(newFeedback);
    alert("Feedback submitted successfully!")
}
}

```

In this code example, we build a dynamic form using reactive forms. Let's take a closer look at each part of the code to see how it works.

- **ratings: string[] = ['1', '2', '3', '4', '5'];** This line of code defines an array of string values, which will be used in the dynamic form to populate the radio buttons.
- **likes: string[] = ['Service', 'Quality', 'Price', 'Ambience', 'Other'];** This line of code defines an array of string values, which will be used in the dynamic form to populate the checkboxes.
- **recommendedOptions: string[] = ['Yes', 'No'];** This line of code defines an array of string values, which will be used in the dynamic form to populate the select menu.
- **get likesArray():** This method is a “get” method and is used to return the likes array.
- **leaveFeedback():** This method is used for processing forms submissions. In the body of this method, we capture the values from each of the form controls and build an object literal with the captured values. Next, we print the values to the browsers console window and then we add the object literal to the preexistingFeedback array.

Rerun the unit tests to ensure they are passing (ng test).

```
22 07 2024 15:33:57.320:INFO [launcher
22 07 2024 15:33:58.288:INFO [Firefox
  zVOhxEV9Fvky0GNtAAAB with id 18697196
LOG: 'Order after adding:', Object{tac
, noOnions: ..., noCilantro: ..., quan
Firefox 128.0 (Mac OS 10.15): Executed
LOG: 'Order after adding:', Object{tac
Firefox 128.0 (Mac OS 10.15): Executed
)
TOTAL: 20 SUCCESS
```

Next, start the server (ng serve) and test the FeedbackComponent's functionality. There should be a web form to leave feedback and underneath the form there should be a running list of the feedback.

Customer Feedback

Rating: 5 Likes: Service Quality Would you recommend us? Yes Comments: Everything was perfect, from the service to the quality of the products.	Rating: 4 Likes: Price Would you recommend us? Yes Comments: Great prices and good service, will definitely reco
---	---

Complete the form below to leave feedback.

Feedback Form

Rate our service

☐ 1

☐ 2

☐ 3

☐ 4

☐ 5

What did you like about our service?

☐ Service

☐ Quality

☐ Price

☐ Ambience

☐ Other

What you recommend us?

Any additional comments?

Leave Feedback

Hands-On 6.1: TDD and Complex Forms

In this hands-on project, you will be extending the rpg-character-builder by building the CreateGuide component. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter's hands-on project.
2. Write the HTML, CSS, and TypeScript code for the CreateGuild component. At minimum, you must include the following:

- a. A reactive or template-driven form to create new guilds. This form must include the following form controls: `guildName` (text), `description` (textarea), `type` (select menu) with options for “Competitive, Casual, Social, Educational,” `acceptTerms` (checkbox), and `notificationPreference` (radio) with options for “Email, SMS, In-App).
 - b. Form Validation:
 - i. All fields are required.
 - ii. Accept terms must be checked.
 - iii. Prevent form submissions if the form is invalid.
 - c. Display the created guilds in a separate section within the HTML. Use a table, unordered list, or cards with columns and rows to display the data.
 - d. Update the navigation in your web application to support bi-direction navigation (navbar links and footer links) to this component.
3. Write unit tests to verify the functionality of the CreateGuild component. You should write the tests before you implement the components functionality, following TDD principles. You may choose three test cases of your own, but they must test the forms functionality.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable component. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- Use a template-driven or reactive form to build your form.
- Use a TypeScript interface for the form model.
- Use a method to handle the form submission.
- Display the submitted data in a different section of the HTML page.
- Add data validation to ensure all fields are required, accepted terms is checked, and submissions are not allowed when the form is invalid.
- Reference the work you completed in the virtual-taco-stand and rpg-character-builder applications for assistance with the form and data validation.

Chapter 7. Input/Output Properties

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Examine the use of Input/Output properties for component communication in Angular.
- Assess the impact of Input/Output properties on the modularity and functionality of Angular applications.
- Test for the correct implementation of Input/Output properties through unit testing.
- Compare the component communication in Angular with other web frameworks.

Input/Output Properties

In this section, we will explore component communications using input/output properties. Input/output properties are part of Angular's core framework and are used for communications between parent and child components. @Input allows data to flow from a parent component to a child component and @Output allows data to flow from child components to parent components. Input/output properties are a core principle in Angular development and are used in a lot of corporate applications.

Continuing with where we left off from the previous chapters virtual-taco-stand project:

1. Open the virtual-taco-stand project and open a new CLI window.
2. Generate a new component named 'order-summary.'
 - a. ng generate component order-summary
3. Open the order-summary.spec.ts file and write unit tests that check: should calculate total price correctly, should display message for empty order, and should display details for each taco in order.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
```

```
import { OrderSummaryComponent } from './order-summary.component';  
import { CommonModule } from '@angular/common';  
import { Order } from '../order/order.component';
```

```
describe('OrderSummaryComponent', () => {  
  let component: OrderSummaryComponent;  
  let fixture: ComponentFixture<OrderSummaryComponent>;
```

```
  beforeEach(async () => {
```

```

    await TestBed.configureTestingModule({
      imports: [OrderSummaryComponent, CommonModule,
OrderSummaryComponent]
    })
    .compileComponents();

    fixture = TestBed.createComponent(OrderSummaryComponent);
    component = fixture.componentInstance;

    const mockOrder = {
      orderId: 999,
      tacos: [
        { id: 3, name: 'Al Pastor', price: 2.5, quantity: 2 },
        { id: 1, name: 'Carnitas', price: 3, quantity: 1, noOnions: true }
      ]
    };

    component.order = mockOrder;

    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should calculate total price correctly', () => {
    const mockOrder: Order = {
      orderId: 1000,
      tacos: [
        { id: 1, name: 'Carnitas', price: 3, quantity: 2 },
        { id: 3, name: 'Al Pastor', price: 2.5, quantity: 1 }
      ]
    };

    component.order = mockOrder;
    expect(component.getTotal()).toEqual(8.5);
  });

  it('should display message for empty order', () => {
    component.order = { orderId: 1001, tacos: [] };
    fixture.detectChanges();
    const compiled = fixture.nativeElement;

```

```

    expect(compiled.querySelector('p').textContent).toContain('No tacos added
to the order yet.');
```

```

});

it('should display details for each taco in the order', () => {
  const mockOrder: Order = {
    orderId: 1002,
    tacos: [
      { id: 1, name: 'Carnitas', price: 3, quantity: 2 },
      { id: 3, name: 'Al Pastor', price: 2.5, quantity: 1 }
    ]
  };

  component.order = mockOrder;
  fixture.detectChanges();

  const compiled = fixture.nativeElement;
  expect(compiled.querySelector('li').textContent).toContain('2x Carnitas');
  expect(compiled.querySelector('li').textContent).toContain('Price per taco:
$3.00');
});
});
```

In this code example, we create three unit tests: should calculate total price correctly, should display message for empty order, and should display details for each taco in the order. Let's take a closer look at each part of the code to see how it works.

- **it('should calculate total price correctly', () => { ... }):** This unit tests defines a mock order object literal to represent placing a new order. Next, the getTotal() method is called to validate the calculated total.
- **it('should display message for empty order', () => { ... }):** This unit test checks if the a message is displayed in the HTML for empty orders.
- **it('should display details for each taco in the order', () => {}):** This unit test defines a mock order object literal and checks if the HTML displays the correct order.

To ensure our tests are failing, run ng tests from a new CLI window.

Passing Test (Green):

1. Move the HTML code from the OrderComponent for displaying a list of orders to the OrderSummaryComponent and write the remaining HTML, CSS, and TypeScript using Angular input properties.

```

import { Component, Input } from '@angular/core';
import { Order } from '../order/order.component';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-order-summary',
  standalone: true,
  imports: [ CommonModule ],
  template: `
<h1>Order Summary</h1>
@if (order.tacos.length > 0) {
  <ul>
    @for (taco of order.tacos; track taco) {
      <li>
        <strong>{{ taco.quantity }}x {{ taco.name }}</strong>
        <br />
        Price per taco: {{ taco.price | currency:'USD':'symbol':'1.2-2' }}
        <br />
        @if (taco.noOnions) {
          No onions
          <br />
        }
        @if (taco.noCilantro) {
          No cilantro
          <br />
        }
      </li>
    }
  </ul>
  <p><strong>Total:</strong> {{ getTotal() | currency:'USD':'symbol':'1.2-2' }}</p>
} @else {
  <p>No tacos added to the order yet.</p>
}
`,
  styles: `
li {
  margin-bottom: 10px;
  padding: 5px;
}
`,
})
export class OrderSummaryComponent {
  @Input() order!: Order;

```

```

    getTotal() {
      return this.order.tacos.reduce((acc, taco) => acc + (taco.price * (taco.quantity
?? 1)), 0);
    }
  }
}

```

In this code example, we move the code from the original OrderComponent to the OrderSummaryComponent. The functionality of this component is roughly identical to what we did in the original OrderComponent.

2. Refactor the OrderComponent by removing the code for the order list and include the code to call the OrderSummaryComponent.

```

export interface Taco {
  id: number;
  name: string;
  price: number;
  noOnions?: boolean;
  noCilantro?: boolean;
  quantity?: number;
}

```

```

export interface Order {
  tacos: Taco[];
  orderId: number;
}

```

```

import { Component, EventEmitter, Output } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { OrderSummaryComponent } from '../order-summary/order-
summary.component';

```

```

@Component({
  selector: 'app-order',
  standalone: true,
  template: `
    <div class="order-form-container">
      <form class="order-form" #tacoOrderForm="ngForm"
      (ngSubmit)="addToOrder();">
        <h1>Complete the form below to place a new order.</h1>

        <fieldset>

```

```

    <legend>My Order</legend>
    <label for="tacoType">Taco Type</label>
    <select name="tacoType" id="tacoType" [(ngModel)]="selectedTacoId"
ngModel>
      @for (taco of tacos; track taco) {
        <option value="{{ taco.id }}">{{ taco.name }}</option>
      }
    </select>

    <label for="qty">Quantity</label>
    <input type="text" id="qty" name="qty" class="qty-input"
[(ngModel)]="quantity" ngModel>

    <div class="customization-section">
      <label>Customize</label>

      <div class="customization-option">
        <input type="checkbox" id="noOnions" name="noOnions"
[(ngModel)]="noOnions" ngModel>
        <label for="noOnions">No Onions</label>
      </div>

      <div class="customization-option">
        <input type="checkbox" id="noCilantro" name="noCilantro"
[(ngModel)]="noCilantro" ngModel>
        <label for="noCilantro">No Cilantro</label>
      </div>
    </div>

    <input type="submit" value="Add to Order" />
  </fieldset>
</form>

  <div class="order-summary">
    <app-order-summary [order]="order"></app-order-summary>
  </div>
</div>
`,
  styles: [`
.order-form-container {
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

```

```
.order-form {  
  flex: 1;  
  margin-right: 20px;  
}  
  
.order-summary {  
  flex: 1;  
}  
  
fieldset {  
  margin-bottom: 20px;  
}  
  
label, select, qty-input {  
  display: block;  
  margin-bottom: 5px;  
}  
  
.qty-input, select, input[type="submit"] {  
  padding: 8px;  
  box-sizing: border-box;  
}  
  
select {  
  width: 100%;  
}  
  
.qty-input {  
  width: 20%;  
}  
  
input[type="submit"] {  
  float: right;  
}  
  
.customization-section {  
  margin-top: 10px;  
}  
  
.customization-option {  
  display: flex;  
  align-items: center;  
  margin-bottom: 5px;
```

```

    }

    input[type="checkbox"] {
        margin-right: 5px;
    }
    /*
    // Removed this from the original styling
    .order-summary li {
        margin-bottom: 10px;
        padding: 5px;
    }
    */
    ,
    ],
    imports: [FormsModule, CommonModule, OrderSummaryComponent]
})
export class OrderComponent {
    tacos: Taco[];
    order: Order;
    selectedTacold: number;
    quantity: number;
    noOnions: boolean = false;
    noCilantro: boolean = false;
    orderTotal: number;

    @Output() orderUpdated = new EventEmitter<Order>();

    constructor() {
        this.tacos = [
            {id: 1, name: "Carnitas Taco", price: 3.25},
            {id: 2, name: "Queso Birria Taco", price: 3.50},
            {id: 3, name: "Al Pastor Taco", price: 3.25},
            {id: 4, name: "Tacos de Lengua", price: 3.50},
            {id: 5, name: "Chicken Taco", price: 3.25},
            {id: 6, name: "Fish Taco", price: 3.25},
            {id: 7, name: "Veggie Taco", price: 3.25},
            {id: 8, name: "Chicharron Taco", price: 3.25},
            {id: 9, name: "Potato Taco", price: 3.25},
            {id: 10, name: "Chorizo Taco", price: 3.25}
        ];

        this.order = { tacos: [], orderId: 0 };
        this.selectedTacold = this.tacos[0].id;
        this.quantity = 1;
    }

```

```

    this.orderTotal = 0;
  }

  addToOrder() {
    const selectedTacoNum = Number(this.selectedTacold);

    const selectedTaco = this.tacos.find(taco => taco.id === selectedTacoNum);

    // random number between 1 and 1000 for order Id no decimal places
    this.order.orderId = Math.floor(Math.random() * 1000) + 1;

    if (selectedTaco !== undefined) {
      const tacoToAdd = {
        id: selectedTaco.id,
        name: selectedTaco.name,
        price: selectedTaco.price,
        noOnions: this.noOnions,
        noCilantro: this.noCilantro,
        quantity: this.quantity
      }

      this.order.tacos.push(tacoToAdd);
      console.log('Order after adding:', this.order);

      this.orderUpdated.emit(this.order);

      this.resetForm();
    } else {
      console.error('Taco not found in the list of available tacos.',
        this.selectedTacold)
    }
  }

  resetForm () {
    if (this.tacos.length > 0) {
      this.selectedTacold = this.tacos[0].id;
    }

    this.quantity = 1;
    this.noOnions = false;
    this.noCilantro = false;
  }
}

```

In this code example, we update the OrderComponent by removing the code we originally used to display a list of orders. The code we added to the OrderSummaryComponent is roughly identical to what we originally had in the OrderComponent.

3. Update the unit tests in the OrderComponent to ensure there are no failures.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';

import { OrderComponent } from './order.component';

describe('OrderComponent', () => {
  let component: OrderComponent;
  let fixture: ComponentFixture<OrderComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [OrderComponent]
    })
    .compileComponents();

    fixture = TestBed.createComponent(OrderComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should add a selected taco to the order with correct customization', () => {
    component.selectedTacoId = 1;
    component.noOnions = true;
    component.noCilantro = false;
    component.quantity = 2;
    component.addToOrder();
    const addedTaco = component.order.tacos[0];

    expect(addedTaco.id).toBe(1);
    expect(addedTaco.quantity).toBe(2);
    expect(addedTaco.noOnions).toBe(true);
    expect(addedTaco.noCilantro).toBe(false);
  });

  /*
```

```

it('should calculate the total price correctly for multiple tacos', () => {
  component.order.tacos = [
    { id: 1, name: 'Taco1', price: 5, quantity: 2},
    { id: 2, name: 'Taco2', price: 6.5, quantity: 3}
  ]

  const totalPrice = component.getTotal();
  expect(totalPrice).toBe(29.5);
});
*/

it('should reset all form fields to their default values after resetForm is called',
() => {
  component.selectedTacoId = 2;
  component.quantity = 3;
  component.noOnions = true;
  component.noCilantro = true;

  component.resetForm();

  expect(component.selectedTacoId).toBe(1);
  expect(component.quantity).toBe(1);
  expect(component.noOnions).toBe(false);
  expect(component.noCilantro).toBe(false);
})
});

```

In this code example, we update the OrderComponent unit tests to ensure they are still passing after we moved the original code in the OrderComponent to the OrderSummaryComponent. Anytime you refactor code in your application, always update your unit tests to ensure they are passing. This is a commonly forgotten step that could potentially invalidate all of your previously passing unit tests. And, depending on your deployment strategy could potentially break the deployment pipeline.

Rerun the unit tests to ensure they passing (ng serve).

```

Firefox 128.0 (Mac OS 10.15): Executed
LOG: 'Order after adding:', Object{ta
WARN: 'NG0956: The configured tracking
creation of the entire collection of
iring destruction and subsequent creat
tc. Please review the "track expressio
es items in a collection. Find more at
Firefox 128.0 (Mac OS 10.15): Executed
WARN: 'NG0956: The configured tracking
creation of the entire collection of
iring destruction and subsequent creat
tc. Please review the "track expressio
Firefox 128.0 (Mac OS 10.15): Executed
)
TOTAL: 23 SUCCESS

```

Start the application (ng serve) to test the functionality of the OrderSummary component. The behavior should be identical to what it was before we added the OrderSummary component to the OrderComponent. If you have errors, it is likely due to a typo or syntax error. Carefully go back through the code in this section to ensure your code matches exactly to what's displayed here.

Hands-On 7.1: TDD and Component Communications

In this hands-on project, you will be extending the rpg-character-builder by updating the CreateCharacter and CreateGuild components to use input/output properties. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter's hands-on project.
2. Update the HTML, CSS, and TypeScript code in the CreateCharacter and CreateGuild components to use input/output properties for the form submitted data (list of created characters and list of created guilds). At minimum, you must include the following:
 - a. A separate component to display the list of newly created characters.
 - b. A separate component to display the list of newly created guilds.
 - c. Input/output properties to link the components created in 5.a. and 5.b.
 - d. Display the data in a table, unordered list, or cards with columns and rows to display the data.

4. Write unit tests to verify the functionality of the input/output properties. You should write the tests before you implement the components functionality, following TDD principles. You may choose three test cases of your own, but they must test the new components functionality. For suggestions, consider the following:
 - e. Test 1 (suggestion): characters in the newly created component are displaying correctly.
 - f. Test 2 (suggestion): guilds in the newly created component are displaying correctly.
 - g. Test 3 (suggestion): should display a message for an empty guild list.
 - h. Test 4 (suggestion): should display a message for an empty character list.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable Player's page. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- Generate two components: `CharacterList` and `GuildList` (ex: `ng generate component character-list`).
- Once you move the code from the original components to the generated components, do not forget to update the original components codebase.
- Update the original components unit tests to ensure they are still passing.
- Use the `virtual-taco-stand` project as a reference point for introducing input/output properties.

Chapter 8. HTTP Client

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Evaluate the use of HTTP requests in Angular.
- Examine data retrieval through HTTP requests in Angular.
- Assess the integration of Angular with external APIs.
- Contrast building versus consuming APIs in full-stack development.

HttpClient

In this section we will take a look at how we can leverage Angular's HttpClient module to make an HTTP request to an API. The HttpClient is a powerful library for handling HTTP requests and responses and is a core principle of any web application. It provides a way for us to communicate with servers and APIs to execute GET, POST, PUT/PATCH, and DELETE operations over the HTTP/HTTPS protocol. Any application that depends on a database or dynamic data will eventually need some type of mechanism to interact with a web API.

Here are some of the key points of the HttpClient module:

1. **Asynchronous Operations:** The HttpClient module performs operations asynchronously, which means the application will continue to run while it waits for a response from either a web server or API.
2. **Observable-based API:** Under the hood, the HttpClient returns an Observable from the RxJS library. This means, data is treated as streams, so you can subscribe, manipulate, and transform the data as needed.
3. **JSON Support:** By default, the HttpClient module treats responses as JSON, which makes it easier to work with complex data structures and it's the most commonly used format in a web service.
4. **Error Handling:** Given the fact that the HttpClient returns an Observable, you can catch and handle errors with minimal effort.

Continuing where we left off from in the virtual-taco-stand project:

1. Open the daily-specials.component.spec.ts file and add unit tests for, getDayOfTheWeek should return a valid day of the week, should handle error when daily special is not found, should update dailySpecial when a daily special is found.

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { HttpClientTestingModule, HttpTestingController } from
 '@angular/common/http/testing';
import { DailySpecialsComponent } from './daily-specials.component';
```

```
describe('DailySpecialsComponent', () => {
  let component: DailySpecialsComponent;
  let fixture: ComponentFixture<DailySpecialsComponent>;
  let httpTestingController: HttpTestingController;
```

```
  beforeEach(async () => {
    await TestBed.configureTestingModule({
```

```

    imports: [DailySpecialsComponent, HttpClientTestingModule]
  })
  .compileComponents();

  httpTestingController = TestBed.inject(HttpTestingController);

  fixture = TestBed.createComponent(DailySpecialsComponent);
  component = fixture.componentInstance;
  fixture.detectChanges();
});

it('should create', () => {
  expect(component).toBeTruthy();
});

it('getDayOfTheWeek should return a valid day of the week', () => {
  const day = component.getDayOfTheWeek();
  expect(['Sunday', 'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday',
'Saturday']).toContain(day);
});

it('should handle error when daily special is not found', () => {
  const expectedMessage = 'No special today. Check back tomorrow!';
  const req = httpTestingController.expectOne(`http://localhost:3000/api/daily-
specials?day=${component.dayOfTheWeek}`);
  req.flush('Special not found', { status: 404, statusText: 'Not Found' });

  expect(component.noSpecialMessage).toEqual(expectedMessage);
});

it('should update dailySpecial when a daily special is found', () => {
  const mockSpecial = {
    day: 'Monday',
    name: 'Carnitas Taco',
    description: 'Slow-cooked pork with fresh cilantro, onions, and salsa on a
corn tortilla.',
    price: 2.60
  }

  const req = httpTestingController.expectOne(`http://localhost:3000/api/daily-
specials?day=${component.dayOfTheWeek}`);
  req.flush(mockSpecial);

  expect(component.dailySpecial).toEqual(mockSpecial);

```

```
});  
});
```

To ensure our tests fail, run `ng test` from a new CLI window.

Passing Test (Green):

1. Open the `daily-specials.component.ts` file and Write the HTML, CSS, and TypeScript code for the `DailySpecialsComponent` to pass the defined unit tests

```
import { CommonModule } from '@angular/common';  
import { HttpClient } from '@angular/common/http';  
import { Component } from '@angular/core';
```

```
@Component({  
  selector: 'app-daily-specials',  
  standalone: true,  
  imports: [CommonModule],  
  template: `  
    @if (dailySpecial) {  
      <div class="dspecial-container">  
        <h1>{{ dailySpecial.day }}'s Daily Special</h1>  
        <div class="dspecial-card">  
          <h3>{{ dailySpecial.name }}</h3>  
          <p>{{ dailySpecial.description }}</p>  
          <p>Price: {{ dailySpecial.price | currency }}</p>  
        </div>  
      </div>  
    } @else {  
      <div class="dspecial-container">  
        <h1>{{ noSpecialMessage }}</h1>  
      </div>  
    }  
  `,  
  styles: `  
    .dspecial-container {  
      display: flex;  
      flex-direction: column;  
      padding: 0;  
    }  
  
    .dspecial-card {  
      margin: 10px;
```

```

    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
    padding: 20px;
    width: 80%;
  }
,
})
export class DailySpecialsComponent {
  dayOfTheWeek: string;
  dailySpecial: any = null;
  noSpecialMessage: string = "";

  constructor(private http: HttpClient) {
    this.dayOfTheWeek = this.getDayOfTheWeek();

    this.http.get(`http://localhost:3000/api/daily-specials?day=${this.dayOfTheWeek}`).subscribe({
      next: (res) => {
        console.log(res);
        this.dailySpecial = res;
      },
      error: (err) => {
        console.error('Error fetching daily special', err);
        if(err.error === 'Special not found') {
          this.noSpecialMessage = 'No special today. Check back tomorrow!';
        } else {
          this.noSpecialMessage = 'Error fetching daily special. Please try again later.';
        }
      }
    });
  }

  getDayOfTheWeek(): string {
    return new Date().toLocaleDateString('en-US', { weekday: 'long' });
  }
}

```

2. To test the implementation of the DailySpecialsComponent, we will need a web API. Open the data-files folder and look for a file named daily-specials.js. Copy this file to your solution. To start the API, open a new CLI window and use Node to execute the script (node daily-specials.js).

Once you've started the API, rerun the unit tests to ensure they are passing (ng test).

```
creation of the entire collection of s
iring destruction and subsequent creat
tc. Please review the "track expressio
es items in a collection. Find more at
Firefox 128.0 (Mac OS 10.15): Executed
WARN: 'NG0956: The configured tracking
creation of the entire collection of s
iring destruction and subsequent creat
tc. Please review the "track expressio
Firefox 128.0 (Mac OS 10.15): Executed
)
TOTAL: 26 SUCCESS
```

Start the application (ng serve) to test the functionality of the DailySpecialsComponent. Clicking on the “Daily Specials” link in the left-hand navigation menu should take you to the DailySpecialsComponent with the current daily special.

Wednesday's Daily Special

AI Pastor Taco

Marinated pork with pineapple, cilantro, and onions on a corn tortilla.

Price: \$2.60

To prove our error handling is working properly, stop the API (Ctrl + C) and refresh the page. You should see a message, indicating there was an error retrieving the daily special.

Error fetching daily special. Please try again later.

Hands-On 8.1: TDD and HttpClient

In this hands-on project, you will be extending the rpg-character-builder by building the CharacterFaction component. You will be applying Test-Driven Development (TDD) principles using Karma and Jasmine.

Instructions:

1. Continue working in the project folder from the previous chapter's hands-on project.
2. Write the HTML, CSS, and TypeScript code for the CharacterFaction component. At minimum, you must include the following:
 - a. Use the HttpClient module to make a call to the provided character-faction.js API (located in the data-files folder) and display the response data in a styled table, unordered list, or cards with columns and rows. The response data from the API call is a list of the available character factions.
 - b. Use the file named character-factions.js (located in the data-files folder) for your API. You can call this service using the following path: `http://localhost:3000/api/character-factions`. And, to execute the script, use `node character-factions.js`.
 - c. Update the navigation in your web application to support bi-direction navigation (navbar links and footer links) to this component.
3. Write unit tests to verify the functionality of the CharacterFaction component. You should write the tests before you implement the components functionality, following TDD principles. Use the following test cases:
 - a. Test 1: should handle errors when data is not found.
 - b. Test 2: should correctly fetch a list of character factions.
 - c. Test 3: should update the characterFactions div when character factions are found.

Grading:

You will earn 20 points for each passing test and 40 points for a styled and presentable Player's page. The total awardable points are 100. If two tests fail, you will earn 40 points for the unit tests. If all unit tests pass, but the page is not styled or presentable, you will earn 60 points.

Hints:

- To start the API, use the following command: `node character-factions.js`. Use a separate terminal window to start the API server.
- The character factions API is listening on port 3000.

- Specify the entire API path in the HttpGet request: localhost:3000/api/character-factions. For example, this.http.get('http://localhost:3000/api/character-factions').subscribe({ ... });
- Display a custom error message in the webpage for server errors (i.e., server down).

Chapter 9. Modules and Deployment

Chapter Overview

Learning Objectives

By the end of this chapter, you should be able to:

- Examine Angular modules and their structure.
- Discover the process of deploying Angular projects.
- Evaluate the role of AppModule and feature modules.
- Deploy projects to GitHub pages effectively.

GitHub Pages Deployment

In this section we will take a look at how to deploy an Angular application to GitHub Pages, using their free static website hosting service. GitHub pages is a service offered by GitHub that allows users to publish static websites on GitHub. Once a website is successfully deployed to GitHub pages you will receive a URL that links to your website. Websites hosted on GitHub pages are accessible at `https://{your_github_username}.github.io/{repository_name}`

Instructions:

TL;DR: [Source](#)

Before you can deploy your project to GitHub pages, you will need to install a third-party package named **angular-cli-ghpages**. This tool is designed to simplify the deployment process for transpiled Angular projects to GitHub pages. At the time of this writing, the latest version of this tool is 2.0.1. This is the version you should be using to complete the deployment steps in this section.

Add the npm package **angular-cli-ghpages** to your solution (npm install package_name@version).

```

λ ng add angular-cli-ghpages@latest
i Using package manager: npm
!! Package has unmet peer dependencies. Adding the package may not succeed.

The package angular-cli-ghpages@latest will be installed and executed.
Would you like to proceed? Yes
✓ Packages successfully installed.
UPDATE angular.json (2821 bytes)

```

Something to consider: Before you can deploy your project to GitHub pages all pending changes must be staged, committed, and pushed to GitHub.

Deploy the project to GitHub pages using the following command: **ng deploy --base-href=/{repository name}/**

```

λ ng deploy --base-href=/ng-test-deploy/
[+] Building "ng-test-deploy"
[+] Build target "ng-test-deploy:build:production"
✓ Browser application bundle generation complete.
✓ Copying assets complete.
✓ Index html generation complete.

Initial Chunk Files | Names | Raw Size | Estimated Transfer Size
main.ac40e5296b55d01f.js | main | 183.21 kB | 49.48 kB
polyfills.e85a456e4270eb4c.js | polyfills | 33.04 kB | 10.65 kB
styles.e4ff914b993ed4a2.css | styles | 1016 bytes | 341 bytes
runtime.3cb0d81895311829.js | runtime | 906 bytes | 518 bytes
you will need to install a third-party
igned to simplify the deployment pro | Initial Total | 218.12 kB | 60.97 kB

Build at: 2023-10-03T18:31:02.619Z - Hash: 30ee2ce1972dd898 - Time: 24019ms
[+] Uploading via git, please wait...
[+] Successfully published via angular-cli-ghpages! Have a nice day!

```

Something to consider: For the command to work you will need to remove the curly braces {} and replace the repository name with the repository name of your project. As you can see from the above image, the project I am deploying has a repository name of ng-test-deploy. If the names do not match, the deployment will fail. Also, do not forget the trailing slash / after the repository name.

Verify the project was deployed to GitHub pages by opening a new browser window and entering the deployment URL. The deployment URL is always your GitHub **username.github.io/repository**. For example, the deployed URL for the ng-test-deploy website is as follows: <https://buwebdev.github.io/ng-test-deploy>

Homes for Cheap

Where you can find the best deals on homes

[Home](#)[Listings](#)[About Us](#)[Contact Us](#)

Our Mission

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur interdum turpis a nisi congue dapibus. Etiam dapibus euismod urna, sed tempus ipsum feugiat nec. Mauris ac lectus ipsum, eu imperdiet dui. Quisque lobortis interdum nibh ac lobortis. Nulla facilisi. Cras portitor mauris ut justo mattis lobortis. Vivamus malesuada libero ut sem euismod eu fringilla ipsum feugiat. Nunc sed vestibulum lorem. Vivamus sit amet metus non urna pharetra feugiat ut ac massa. Praesent laoreet fringilla enim, nec dapibus metus porta eget. Donec interdum arcu vel nunc consectetur eu rhoncus libero viverra. Nam eleifend tortor in metus accumsan ultrices.

Sed ante purus, malesuada eget varius non, consequat eu purus. Nulla rhoncus ultrices urna nec bibendum. Nulla ipsum nibh, tempus a hendrerit id, hendrerit in urna. Morbi condimentum felis sed urna pulvinar rutrum. Mauris condimentum porta nunc, quis semper risus vulputate a. Praesent sollicitudin sem vel lacus vestibulum eleifend vulputate ligula consectetur. Duis nulla orci, commodo a dapibus sed, scelerisque vel justo. Vivamus sit amet ligula lacus, pharetra tempor massa. Aliquam vel est vitae nulla accumsan cursus. Vivamus lectus neque, euismod in convallis sit amet, adipiscing quis nunc. Quisque nec turpis at neque faucibus malesuada. Vestibulum aliquam interdum ultricies. Nam quis urna mi, vel bibendum nibh. Pellentesque ullamcorper nunc ut enim tempus rutrum. Maecenas lacus magna, lobortis ac imperdiet eu, lobortis eu elit. Duis vitae ipsum et quam rhoncus portitor vitae sit amet nunc. Sed non magna eu nisl fringilla lacinia. Vestibulum ligula eros, imperdiet et adipiscing tempor, vestibulum id lorem. Donec semper sapien eu magna tristique cursus.

Featured Listings



Beautiful blue one story home for sale!



Downtown living? Check! Enjoy the nightlife with a spacious downtown condo!



Want a home on the lake? We have you covered. Check out this amazing deal!

Something to consider: If the deployment fails, go to your GitHub repository, select Settings, and Pages. Look for a section called Build and Deployment and verify that the Deploy from a branch option is selected and under Branch make sure gh-pages is selected. Also, each time you make changes to your project you will have to re-run the above deployment steps. For Angular projects, changes are not automatically updated when the code is pushed to GitHub. The entire project has to be transpiled through the Ivy compiler (i.e., ng deploy).

References

Angular. (2024). Angular. What is Angular: <https://angular.dev/overview>

Copilot. (n.d.). OpenAI. *Microsoft Copilot*. computer software. Retrieved December 19, 2023, from <https://www.microsoft.com/en-us/microsoft-copilot>.